LittleLumin
AI Powered Parent Assistant

Mini Project Report

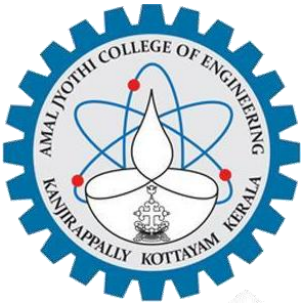
Submitted by

ARUN BIJU THOMAS

Reg. No.: AJC25MCA-2019

In Partial fulfillment for the Award of the Degree of

MASTER OF COMPUTER APPLICATIONS (MCA)



AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

[Approved by AICTE, Accredited by NAAC with A+, Accredited by NBA.
Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2025-2027

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

CERTIFICATE

This is to certify that the Project report titled "**LITTLELUMIN"** is the bona fide work of **ARUN BIJU THOMAS (Regno: AJC25MCA-2019)** carried out in partial fulfillment of the requirements for the award of the **Degree of Master of Computer Applications** at **Amal Jyothi College of Engineering Autonomous, Kanjirappally.** The project was undertaken during the period from **July 01, 2026** to **October 27, 2026**

Ms. Nimmy Francis
Internal Guide

Ms. Meera Rose Thomas
Coordinator

Dr. Bijimol TK
Head of the Department

DECLARATION

I hereby declare that the project report **"LITTLELUMIN"** is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, towards the partial fulfilment of the requirements for the award of the **Master of Computer Applications (MCA)** during the period from **July 01, 2026** to **October 27, 2026**.

Date: 27 / 10 2026
KANJIRAPPALLY

ARUN BIJU THOMAS
Reg: AJC25 MCA-2019

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Dr. Bijimol T K.** for helping us. I extend my whole hearted thanks to the project coordinator **Coordinator Name** for his valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Guide Name** for his inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

ARUN BIJU THOMAS

ABSTRACT

LittleLumin: An AI-Powered Parenting Guidance and Early Childhood Development Platform

LittleLumin is an AI-powered parenting guidance and early childhood development platform designed to help parents support their children's growth through personalized guidance, age-appropriate activities, and interactive stories. The system aims to encourage children's learning and development while improving parent-child engagement.

The application is developed using Flutter and Dart, with FastAPI and Python for backend services, Firebase Authentication and Cloud Firestore for user management and data storage, and Google Gemini API for AI-powered content generation. The backend is deployed on Render. LittleLumin aims to make parenting resources more accessible and support a positive early childhood learning experience.

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	01
1.1	PROJECT OVERVIEW	02
1.2	PROJECT SPECIFICATION	02
2	SYSTEM STUDY	04
2.1	INTRODUCTION	05
2.2	LITERATURE REVIEW	05
2.3	DRAWBACKS OF EXISTING SYSTEM	06
2.4	PROPOSED SYSTEM	06
2.5	ADVANTAGES OF PROPOSED SYSTEM	07
3	REQUIREMENT ANALYSIS	08
3.1	FEASIBILITY STUDY	09
3.1.1	ECONOMICAL FEASIBILITY	09
3.1.2	TECHNICAL FEASIBILITY	10
3.1.3	BEHAVIORAL FEASIBILITY	10
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	10
3.1.5	GEOTAGGED PHOTOGRAPH	12
3.2	SYSTEM SPECIFICATION	12
3.2.1	HARDWARE SPECIFICATION	12
3.2.2	SOFTWARE SPECIFICATION	12
3.3	SOFTWARE DESCRIPTION	13
3.3.1	FLUTTER	13
3.3.2	FASTAPI AND PYTHON	13
3.3.3	FIREBASE	13
3.3.4	GOOGLE GEMINI API	13
4	SYSTEM DESIGN	14
4.1	INTRODUCTION	15
4.2	UML DIAGRAM	15
4.2.1	USE CASE DIAGRAM	15
4.2.2	SEQUENCE DIAGRAM	17
4.2.3	STATE CHART DIAGRAM	18
4.2.4	ACTIVITY DIAGRAM	19

4.2.5	CLASS DIAGRAM	21
4.2.6	OBJECT DIAGRAM	22
4.2.7	COMPONENT DIAGRAM	22
4.2.8	DEPLOYMENT DIAGRAM	23
4.2.9	COLLABORATION DIAGRAM	24
4.3	USER INTERFACE DESIGN USING FIGMA	25
4.4	DATABASE DESIGN	27
5	SYSTEM TESTING	46
5.1	INTRODUCTION	47
5.2	TEST PLAN	47
5.2.1	UNIT TESTING	48
5.2.2	INTEGRATION TESTING	48
5.2.3	VALIDATION TESTING	48
5.2.4	USER ACCEPTANCE TESTING	49
5.2.5	AUTOMATION TESTING	49
5.2.6	SELENIUM TESTING	50
6	IMPLEMENTATION	64
6.1	INTRODUCTION	65
6.2	IMPLEMENTATION PROCEDURE	65
6.2.1	USER TRAINING	65
6.2.2	TRAINING ON APPLICATION SOFTWARE	66
6.2.3	SYSTEM MAINTENANCE	66
7	CONCLUSION & FUTURE SCOPE	68
7.1	CONCLUSION	69
7.2	FUTURE SCOPE	69
8	BIBLIOGRAPHY	70
9	APPENDIX	72
9.1	SAMPLE CODE	73
9.2	SCREEN SHOTS	87
9.3	GIT LOG	89

List of Abbreviations

- AI – Artificial Intelligence
- API – Application Programming Interface
- ADHD – Attention-Deficit/Hyperactivity Disorder
- VSMS – Vineland Social Maturity Scale
- SQ – Social Quotient
- SA – Social Age
- UI – User Interface
- UML – Unified Modeling Language
- JSON – JavaScript Object Notation
- REST – Representational State Transfer
- CRUD – Create, Read, Update, Delete
- UAT – User Acceptance Testing

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

LittleLumin is an AI-powered parenting guidance and early childhood development platform designed to support parents in understanding and encouraging their children's growth. The system provides parenting guidance, age-appropriate activities, and interactive story generation through an intuitive application. By integrating artificial intelligence, LittleLumin aims to make developmental resources more accessible and promote engaging learning experiences for children.

The platform uses Flutter and Dart for the frontend, FastAPI and Python for backend services, and Firebase Authentication and Cloud Firestore for user authentication and data management. Google Gemini API enables AI-powered activity and story generation. LittleLumin aims to assist parents in creating supportive learning environments and encourage positive early childhood development through accessible and interactive digital resources.

1.2 PROJECT SPECIFICATION

The LittleLumin system consists of the following main modules and functionalities:

Parent Module Functionalities

1. **User Registration and Login:** Allows parents to create accounts and securely access the application.
2. **Child Profile Management:** Maintains relevant information about children to support personalized developmental activities.
3. **Developmental Assessment:** Provides a questionnaire-based assessment using the Vineland Social Maturity Scale (VSMS) to evaluate children's social maturity across different developmental domains.
4. **Developmental Progress Monitoring:** Calculates Social Age (SA) and Social Quotient (SQ) from assessment results to help parents understand their child's developmental progress.
5. **Parenting Guidance:** Provides AI-assisted guidance and resources to help parents understand developmental needs and support their children's growth.
6. **Activity Recommendations:** Offers suitable activities that encourage children's cognitive, social, communication, motor, and self-help skills.
7. **Community Support:** Supports access to relevant community events and developmental resources, subject to the availability of the corresponding module.

Child Module Functionalities

1. **Interactive Learning:** Provides engaging learning experiences through stories, puzzles, and developmental activities.
2. **Story Mode:** Generates or presents stories to encourage imagination, language development, and comprehension.
3. **Screen-Free Activities:** Recommends practical activities that encourage children to learn through physical interaction with their surroundings.
4. **Handwriting Recognition:** Supports handwriting-related learning through recognition and analysis features, where implemented.
5. **Speech Interaction:** Supports speech-based interaction and command recognition, where available in the application.

Artificial Intelligence Functionalities

1. **AI-Based Activity Generation:** Uses Google Gemini API to generate relevant activities based on user requirements.
2. **AI-Based Story Generation:** Generates engaging stories to support children's learning and imagination.
3. **Personalized Developmental Support:** Aims to recommend activities according to assessment results and individual developmental needs.
4. **Adaptive Learning Support:** Supports the planned adjustment of activities according to children's performance and developmental progress.

System Functionalities

1. **Authentication and Data Management:** Uses Firebase Authentication and Cloud Firestore for user authentication and application data management.
2. **Backend Processing:** Uses FastAPI and Python to handle application requests and AI-related operations.
3. **Application Interface:** Uses Flutter and Dart to provide an interactive mobile application.
4. **Cloud Deployment:** Deploys backend services through Render to enable remote access to AI functionalities.

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

In the modern digital era, technology plays an important role in improving early childhood development and parenting support. LittleLumin is an AI-powered platform designed to help parents access developmental guidance, learning activities, and interactive stories for their children. Traditional parenting resources may not always provide personalized guidance or suitable activities based on individual developmental needs. This project aims to address these challenges by introducing an interactive digital platform that combines artificial intelligence with early childhood learning support.

LittleLumin enables parents to access AI-generated activities, stories, and parenting resources through a user-friendly application. By integrating Flutter, FastAPI, Firebase, and Google Gemini API, the system provides an accessible platform for generating learning content and supporting parents in their children's developmental journey.

The primary goal of this project is to improve access to parenting guidance and encourage positive early childhood development through technology.

2.2 LITERATURE REVIEW

The World Health Organization (WHO), UNICEF, and the World Bank Group introduced the *Nurturing Care Framework* in 2018, emphasizing the importance of responsive caregiving, early learning opportunities, health, nutrition, and safety in childhood development. The framework highlights the role of parents and caregivers in creating supportive environments for children's growth. These principles provide a foundation for developing digital platforms such as LittleLumin that support parents through developmental guidance and learning activities.

A scoping review published in 2025, titled *Parenting in the Digital Age*, examined digital interventions for early childhood parenting in low- and middle-income countries. The review identified the potential of digital platforms to improve parenting practices and provide accessible developmental resources, while also highlighting the need for stronger evidence regarding their effectiveness. This supports the importance of designing accessible and reliable digital parenting solutions.

Research on artificial intelligence in child development monitoring, published in 2024, reviewed the use of AI for developmental assessment and monitoring. The study identified promising applications in cognitive, social, and language development but noted that many existing approaches require further clinical evaluation. Based on these findings, LittleLumin integrates AI-

powered activity and story generation to support early childhood learning and provide accessible resources for parents. The platform is intended as a supportive tool rather than a replacement for professional developmental assessment.

References for your bibliography

1. **World Health Organization. (2018).** Nurturing Care for Early Childhood Development.
2. **Jäggi et al. (2025). Parenting in the Digital Age:** A Scoping Review of Digital Early Childhood Parenting Interventions in Low- and Middle-Income Countries.
3. **Artificial Intelligence in Child Development Monitoring:** A Systematic Review on Usage, Outcomes and Acceptance. (2024).

2.3 DRAWBACKS OF THE EXISTING SYSTEM

1. **Limited Personalization:** Traditional child-development applications often provide generic activities that may not suit each child's individual developmental needs.
2. **Limited Progress Tracking:** Monitoring a child's development across multiple skill areas can be difficult without centralized progress records and meaningful analytics.
3. **Lack of AI-Based Guidance:** Conventional systems may lack intelligent activity generation and personalized recommendations based on a child's performance.
4. **Limited Parental Support:** Parents may not receive sufficient guidance on selecting suitable developmental activities or supporting their child's learning at home.
5. **Fragmented Information:** Activity records, feedback, developmental observations, and progress reports may be maintained separately, making them difficult to review together.
6. **Difficulty Identifying Developmental Needs:** Without structured assessments and continuous progress monitoring, recognizing areas that may require additional attention can be challenging.
7. **Limited Specialist Collaboration:** Existing solutions may not provide an integrated way for parents and child-development specialists to review activities, feedback, and progress.

2.4 PROPOSED SYSTEM

The proposed system, LittleLumin, is an AI-powered parenting guidance and early childhood development platform designed to support parents in addressing their children's developmental and learning needs. The platform provides AI-generated activities, interactive stories, and parenting

guidance through a user-friendly mobile application. By integrating artificial intelligence, it aims to make developmental resources more accessible and encourage meaningful learning experiences for children.

The system uses Google Gemini API to generate activities and stories based on user requirements. Firebase Authentication supports user authentication, while Cloud Firestore manages application data. The frontend is developed using Flutter and Dart, with FastAPI and Python handling backend operations. The backend is deployed on Render to provide access to AI services. LittleLumin aims to simplify access to parenting resources, encourage parent-child interaction, and support a positive early childhood learning experience through an interactive digital platform.

2.5 ADVANTAGES OF PROPOSED SYSTEM

- **AI-Powered Guidance:** Provides accessible parenting guidance and developmental resources.
- **Personalized Activities:** Supports the generation of activities based on user requirements.
- **Interactive Story Generation:** Creates engaging stories that encourage children's imagination and learning.
- **Improved Accessibility:** Makes parenting resources available through a mobile application.
- **User-Friendly Interface:** Provides a simple and interactive interface developed using Flutter.
- **Efficient Data Management:** Uses Firebase services for authentication and application data storage.
- **Scalability:** Uses cloud-based backend services that can be extended to support additional features.
- **Enhanced Parent-Child Engagement:** Encourages parents to participate in children's learning and developmental activities.
- **Future Expandability:** Provides a foundation for integrating additional developmental assessment and learning features.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

Feasibility is defined as the practical extent to which a project can be performed successfully. To evaluate feasibility, a feasibility study is performed, which determines whether the solution considered to accomplish the requirements is practical and workable in the software. Information such as resource availability, cost estimation for software development, benefits of the software to the organization after it is developed and cost to be incurred on its maintenance are considered during the feasibility study. The objective of the feasibility study is to establish the reasons for developing the software that is acceptable to users, adaptable to change and conformable to established standards. Various other objectives of feasibility study are listed below:

- To analyse whether the software will meet organizational requirements
- To determine whether the software can be implemented using the current technology and within the specified budget and schedule
- To determine whether the software can be integrated with other existing software.

If a system does not support the business objectives, it has no real value to the business. While this may seem obvious, many organisations develop systems which do not contribute to their objectives either because they don't have a clear statement of these objectives, because they fail to define the business requirements for the system or because other political or organisation factors influence the system procurement

3.1.1 Economical Feasibility

Economic feasibility evaluates the cost involved in developing, deploying, and maintaining a software system. It helps determine whether the project can be completed within the available budget using affordable resources and technologies. The analysis considers development expenses, software requirements, hosting charges, and ongoing maintenance costs.

LittleLumin is economically feasible because it uses technologies such as Flutter, Python, FastAPI, and Firebase, which provide accessible development tools and free or low-cost service options. Google Gemini API and Render offer usage-based or limited free-tier options, depending on their current plans. Since the project is developed using readily available tools and cloud services, the initial development cost can be minimized. Additional expenses may arise from increased API usage, hosting requirements, and future maintenance. Therefore, LittleLumin can be developed and maintained at a relatively low

cost, making it economically feasible for an academic project.

3.1.2 Technical Feasibility

Technical feasibility evaluates whether the required technologies, tools, and technical resources are available to develop and operate a software system. It considers the compatibility of the technologies, development requirements, integration capabilities, and the availability of suitable hardware and software.

LittleLumin is technically feasible because it uses established technologies such as Flutter and Dart for frontend development, FastAPI and Python for backend services, Firebase Authentication and Cloud Firestore for authentication and data management, and Google Gemini API for AI-powered content generation. Render is used to deploy the backend services. These technologies provide the necessary tools to develop, integrate, and maintain the application. The system can be developed using a standard computer and tested on compatible mobile devices. Therefore, the availability of these technologies and development resources makes LittleLumin technically feasible.

3.1.3 Behavioral Feasibility

Behavioral feasibility evaluates how easily users can understand, accept, and adapt to a proposed system. It considers user interaction, ease of use, learning requirements, and the impact of the system on existing practices. A system is considered behaviorally feasible if users can operate it with minimal difficulty and it meets their needs effectively.

In the case of LittleLumin, the application is designed to provide parents with a simple and user-friendly interface for accessing parenting guidance, AI-generated activities, and interactive stories. The mobile-based platform enables users to access these resources conveniently without requiring advanced technical knowledge. The use of familiar application features and automated content generation helps simplify user interaction. Therefore, LittleLumin is behaviorally feasible, as it aims to provide an accessible and convenient experience for parents and caregivers.

3.1.4 Feasibility Study Questionnaire

A feasibility study questionnaire was prepared to collect information from parents, young adults, and child development professionals to understand the challenges associated with parenting and early childhood development. The questionnaire focused on identifying the difficulties faced by parents, the need for personalized developmental activities, and the usefulness of an AI-powered parenting guidance platform. The responses help identify user requirements and evaluate the relevance of the proposed LittleLumin system.

Questionnaire for Data Collection

Category 1: Working Parents

1. What is your age, and how many children do you have?
2. What is the single biggest challenge you face balancing work and parenting?
3. Would you use an app that gives you ready-made, personalized activities to do with your child in just 10–15 minutes?
4. What is the ONE feature such an app must have for you to use it daily?

Category 2: Stay-at-Home Parents

1. What is your age, and how many children do you have?
2. What is the most challenging part of being a stay-at-home parent?
3. Would you use an app that suggests new, personalized activities to do with your child every day?
4. What would stop you from using a parenting app like this?

Category 3: Expecting Parents

1. What is your age, and is this your first pregnancy?
2. What scares you most about becoming a parent?
3. Would you use an app that guides you on age-appropriate activities for your child from birth?
4. What is the ONE thing you wish such an app would do for you?

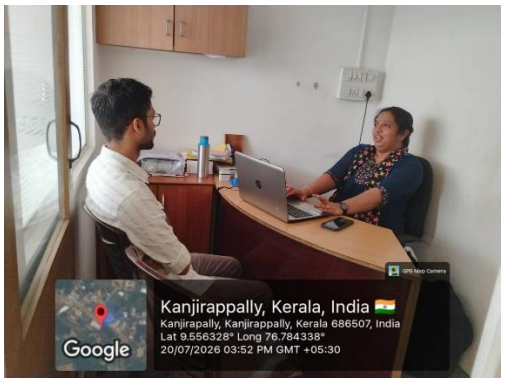
Category 4: Occupational Therapists and Child Specialists

1. What is your profession, and how many years of experience do you have?
2. What is the most common concern parents bring to you about their child's development?
3. Would you recommend a platform that generates personalized, therapeutic-style activities for parents to do at home?
4. What is the ONE feature such a platform must have to be clinically useful for you?

Category 5: Future Parents (Young Adults)

1. What age group do you belong to?
2. What part of becoming a parent feels most unknown or heavy to you right now?
3. Would you want a quiet, non-judgmental guide to help you through those moments?
4. What is one thing you wish such a guide would teach you before you become a parent?

3.1.5 Geotagged Photographs



3.2 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

Processor - Intel Core i5 or equivalent

RAM – 8 GB or above

Hard Disk – SSD with sufficient storage space

Mobile Device – Android smartphone for application testing and usage

3.2.2 Software Specification

Front End – Flutter (Dart)
Back End – FastAPI (Python)
Database – Firebase Cloud Firestore
Client Platform – Android
Operating System – Windows 10 or above for development
Technologies Used – Flutter, Dart, Python, FastAPI, Firebase Authentication, Cloud Firestore, Google Gemini API, REST API, and JSON.

APIs Used

- Google Gemini API – AI-powered activity and story generation.
- Firebase Authentication – User authentication.
- Cloud Firestore – Application data storage and retrieval.

AI Functionalities

- AI-based activity generation.
- AI-based story generation.

- AI-powered parenting guidance.

3.3 SOFTWARE DESCRIPTION

Software description explains the technologies and tools used to develop the LittleLumin mobile application. The system integrates frontend development, backend processing, cloud database services, and artificial intelligence to provide parenting guidance, AI-generated activities, and interactive stories. The major technologies used in the development of LittleLumin are Flutter, FastAPI, Firebase, and Google Gemini API.

3.3.1 FLUTTER

Flutter is an open-source user interface development framework used to build cross-platform applications from a single codebase. It uses the Dart programming language and provides widgets for creating interactive and responsive user interfaces. In LittleLumin, Flutter is used to develop the mobile application interface, allowing parents to access parenting guidance, generated activities, and interactive stories through an Android device.

3.3.2 FASTAPI AND PYTHON

Python is a high-level programming language known for its simplicity and extensive library support. FastAPI is a Python framework used to develop web APIs efficiently. In LittleLumin, FastAPI handles backend operations and connects the mobile application with AI services. It processes requests for generating activities and stories using the Google Gemini API and returns the generated content to the application.

3.3.3 FIREBASE

Firebase is a cloud-based application development platform that provides services for authentication, data storage, and application management. LittleLumin uses Firebase Authentication to manage user authentication and Cloud Firestore to store and retrieve application data. These services support user access and cloud-based data management within the mobile application.

3.3.4 GOOGLE GEMINI API

Google Gemini API provides access to Google's generative artificial intelligence models through API requests. It enables applications to generate text-based responses based on user inputs and instructions. In LittleLumin, the Gemini API is integrated with the FastAPI backend to generate age-appropriate activities and interactive stories, helping parents access engaging content to support children's learning and development.

CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

System design describes the overall structure and organization of the LittleLumin mobile application. It explains how the different components interact to provide parenting guidance, AI-generated activities, and interactive stories. The design includes the application interface, backend services, database, and artificial intelligence integration.

LittleLumin uses Flutter and Dart for mobile application development, FastAPI and Python for backend processing, Firebase for authentication and data management, and Google Gemini API for AI-powered content generation. UML diagrams are used to represent the system's structure, behaviour, and interactions. Database design and user interface design further explain how application data is managed and how users interact with the system.

4.2 UML DIAGRAM

Unified Modeling Language (UML) is a standard visual modeling language used to represent the structure and behaviour of a software system. UML diagrams help explain the interactions between users and system components, the flow of activities, and the relationships between different parts of an application.

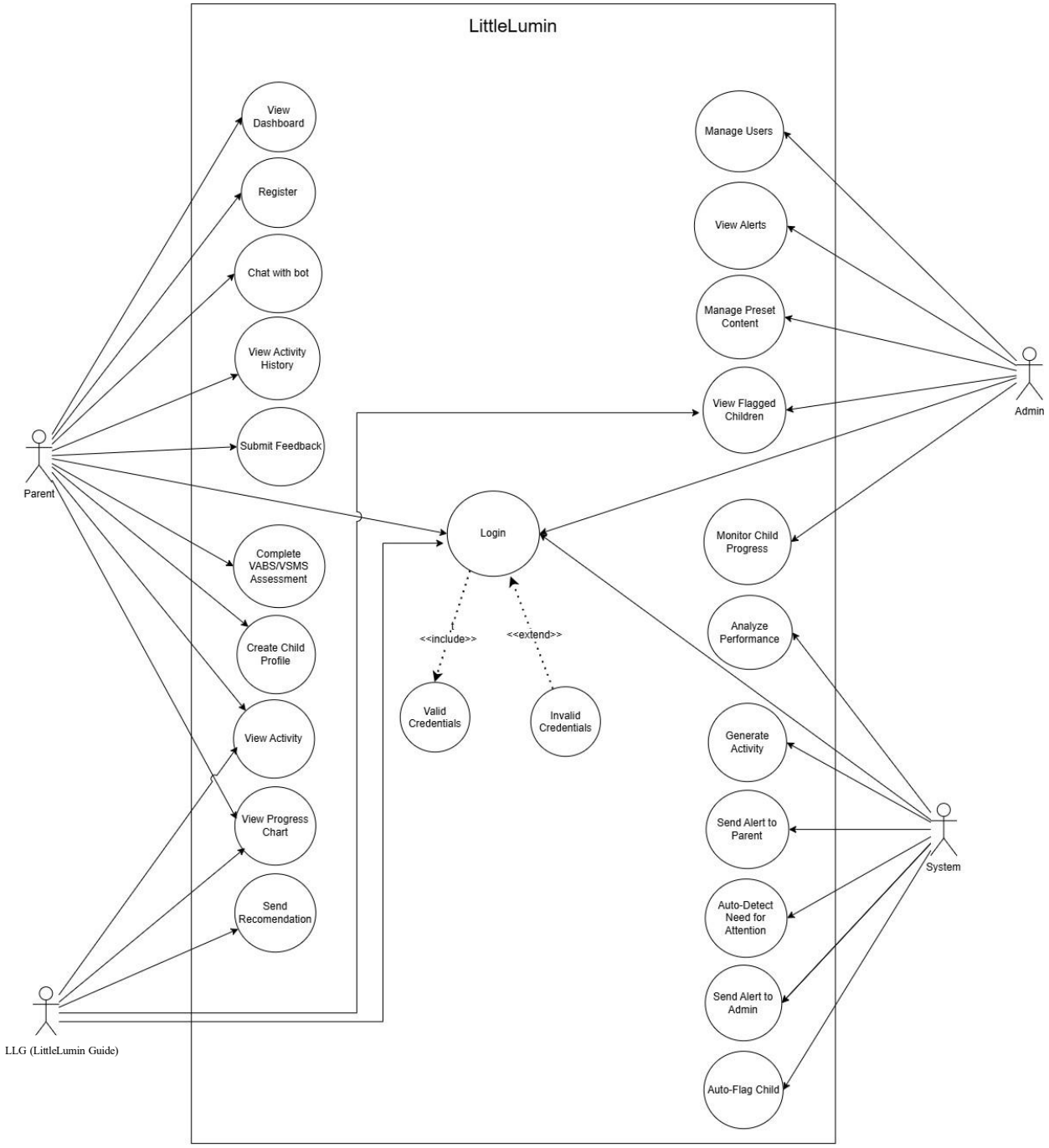
In LittleLumin, UML diagrams are used to represent the system's functionality, workflows, and architecture. The diagrams include the Use Case Diagram, Sequence Diagram, State Chart Diagram, Activity Diagram, Class Diagram, Object Diagram, Component Diagram, Deployment Diagram, and Collaboration Diagram. These diagrams provide a clear understanding of the design and operation of the LittleLumin mobile application.

4.2.1 USE CASE DIAGRAM

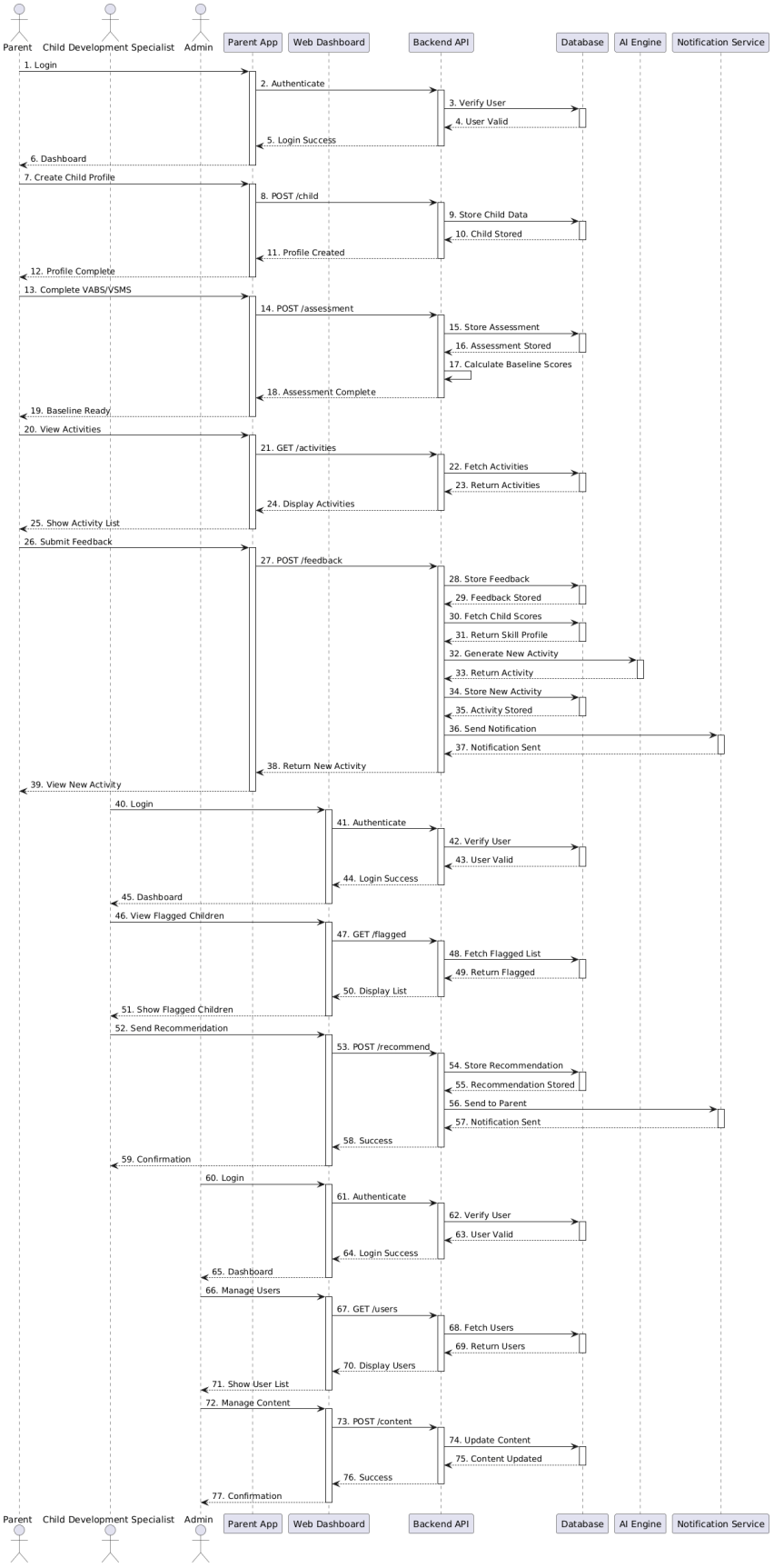
The Use Case Diagram represents the interactions between the users and the LittleLumin system. It illustrates the main functionalities available to parents, the LittleLumin Guide (LLG), administrators, and the system. Parents can register, log in, create child profiles, complete VABS/VSMS assessments, view activities and progress charts, access the chatbot, submit feedback, and receive recommendations. The LittleLumin Guide can view activity information, monitor progress, and provide recommendations.

The administrator manages users, views alerts, manages preset content, and reviews flagged children. The system supports functions such as generating activities, analysing performance,

sending notifications, identifying children who may require attention, and auto-flagging cases for review. The diagram provides an overview of the functional requirements and shows how each actor interacts with the application.



4.2.2 SEQUENCE DIAGRAM



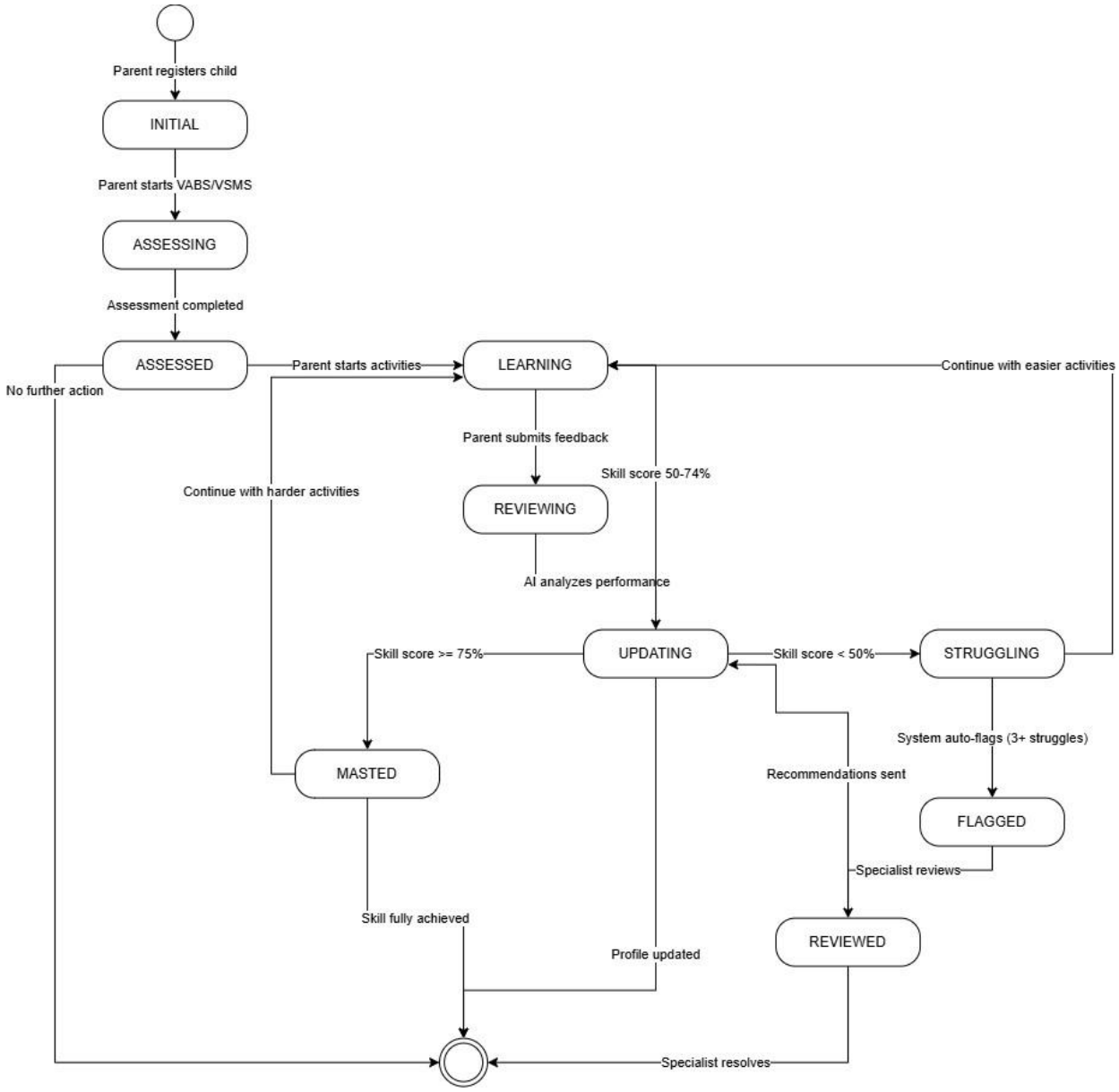
The Sequence Diagram illustrates the sequence of interactions between the users and the different components of the LittleLumin system over time. It represents the communication between the parent, child development specialist, administrator, mobile application, web dashboard, backend API, database, AI engine, and notification service.

The process begins when a parent logs in and the system verifies the user's credentials. The parent can then create a child profile, complete the VABS/VSMS assessment, view activities, and submit feedback. The backend processes these requests, stores and retrieves relevant information from the database, and communicates with the AI engine to generate activities and recommendations. The notification service sends alerts when required. The diagram also illustrates administrator operations, including reviewing flagged children, managing users, and updating content. It provides a clear representation of the order in which system components communicate to perform the required functions.

4.2.3 State Chart Diagram

The State Chart Diagram represents the different states of a child's developmental learning process in the LittleLumin system. It illustrates how the system changes states
based on assessment results, activity performance, and feedback submitted by parents.

The process begins with child registration, followed by the VABS/VSMS assessment. After the assessment is completed, the child enters the assessed state and can begin learning activities. Based on the child's performance and feedback, the system updates the skill profile and determines whether the child has mastered a skill or is experiencing difficulty. When repeated struggles are identified, the child may be flagged for specialist review, and recommendations can be sent to the parent. The diagram illustrates the transitions between assessment, learning, reviewing, updating, and intervention states until the process is completed.

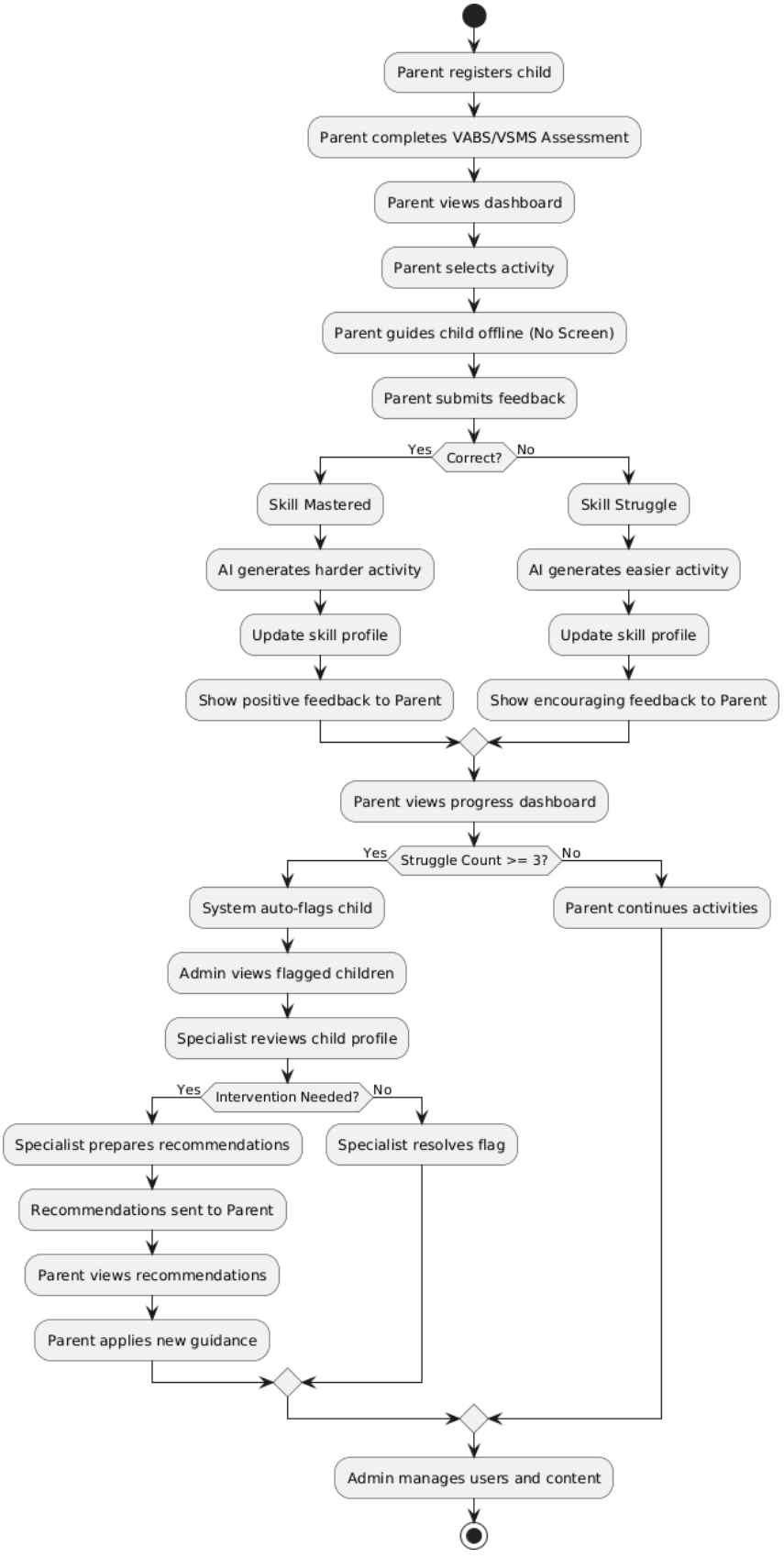


4.2.2 Activity Diagram

The Activity Diagram represents the workflow of the LittleLumin system, illustrating the sequence of activities performed by parents, the system, administrators, and child development specialists. It describes the process from child registration and developmental assessment to activity completion, progress monitoring, and recommendations.

The process begins when a parent registers a child and completes the VABS/VSMS assessment. The parent then selects an activity and guides the child through it without a screen. After submitting feedback, the system evaluates the child's performance and generates suitable activities based on the results. The child's skill profile and progress dashboard are updated accordingly. If repeated

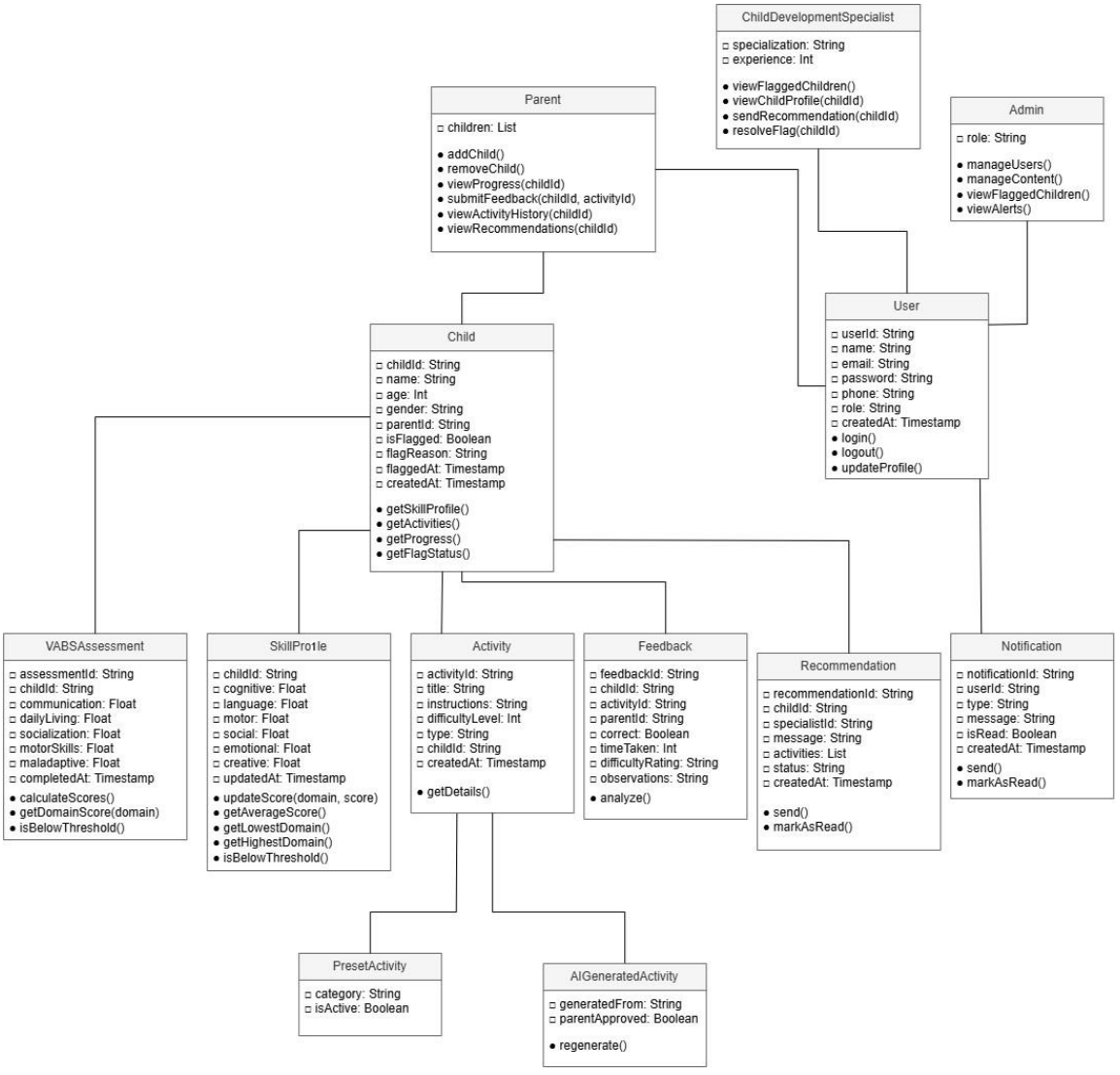
struggles are identified, the system flags the child for review by an administrator and specialist. Based on the review, recommendations may be sent to the parent for further guidance. The diagram illustrates the complete workflow and decision-making process within the application.



4.2.3 Class Diagram

The Class Diagram represents the static structure of the LittleLumin system by illustrating its classes, attributes, methods, and relationships. It provides an overview of the main entities involved in managing user accounts, child profiles, developmental assessments, learning activities, progress tracking, and recommendations.

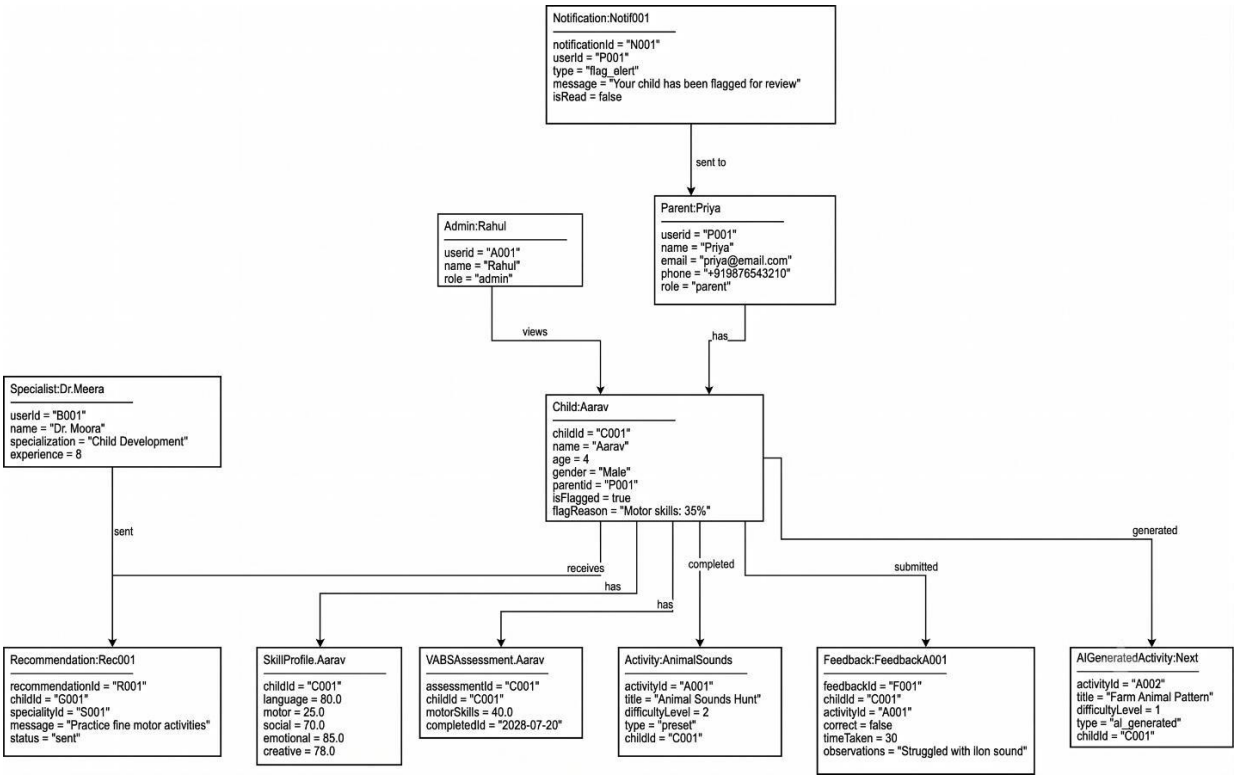
The diagram includes classes such as Parent, Child, Assessment, Activity, SkillProfile, Feedback, Recommendation, and Notification. These classes represent the data and operations required for the application's functionality. Their relationships show how parents manage child profiles, how assessments support developmental analysis, and how activity performance and feedback contribute to progress monitoring. The Class Diagram helps explain the organization of the system and provides a structural view of how its components are related.



4.2.4 Object Diagram

The Object Diagram represents a snapshot of the LittleLumin system at a particular point in time. It illustrates instances of classes and their relationships, showing how individual objects are connected within the application.

In LittleLumin, the Object Diagram provides an example of how objects related to parents, children, assessments, activities, skill profiles, and feedback can interact. It helps visualize the relationships between specific instances of these entities and demonstrates how information is organized during application execution. The diagram complements the Class Diagram by presenting an instance-level view of the system structure.

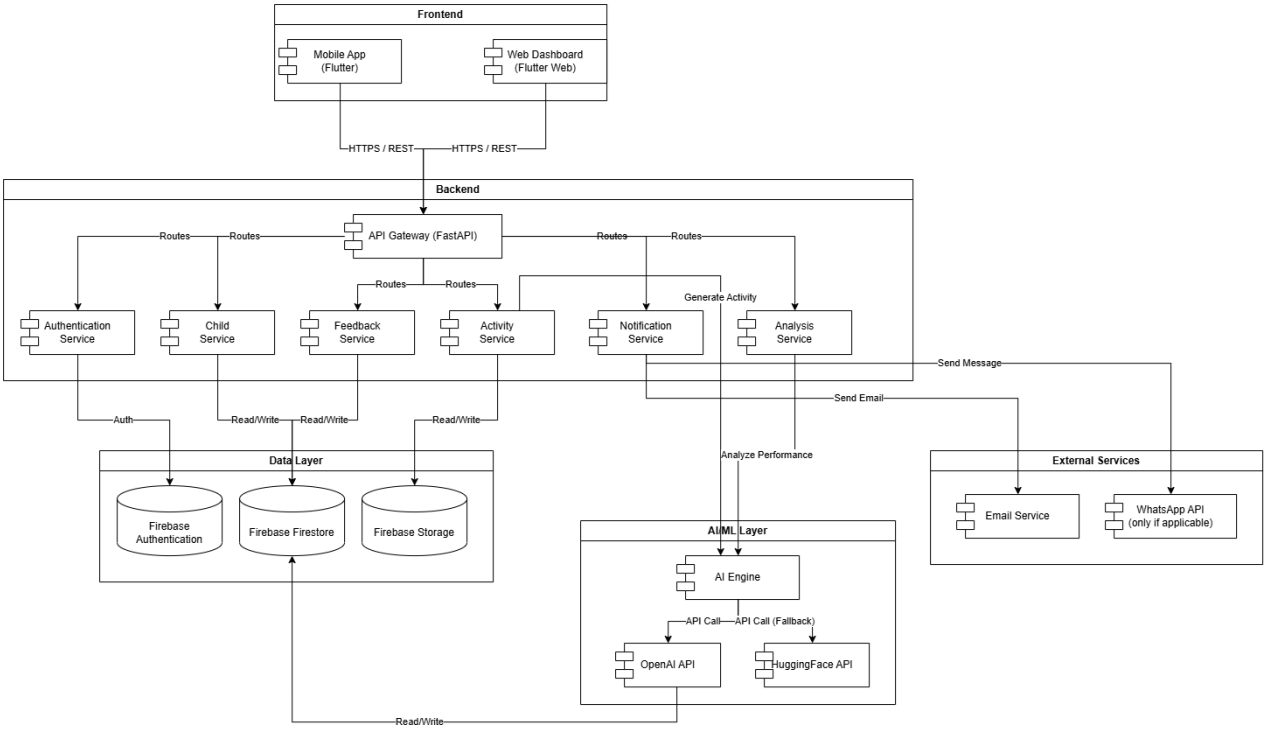


4.2.5 Component Diagram

The Component Diagram represents the major software components of the LittleLumin system and illustrates how they interact with one another. It provides an overview of the system architecture and shows the organization of the application into separate functional components.

LittleLumin consists of a mobile application interface, backend services, database services, and artificial intelligence integration. The Flutter-based mobile application communicates with the FastAPI backend through API requests. The backend interacts with Firebase services for

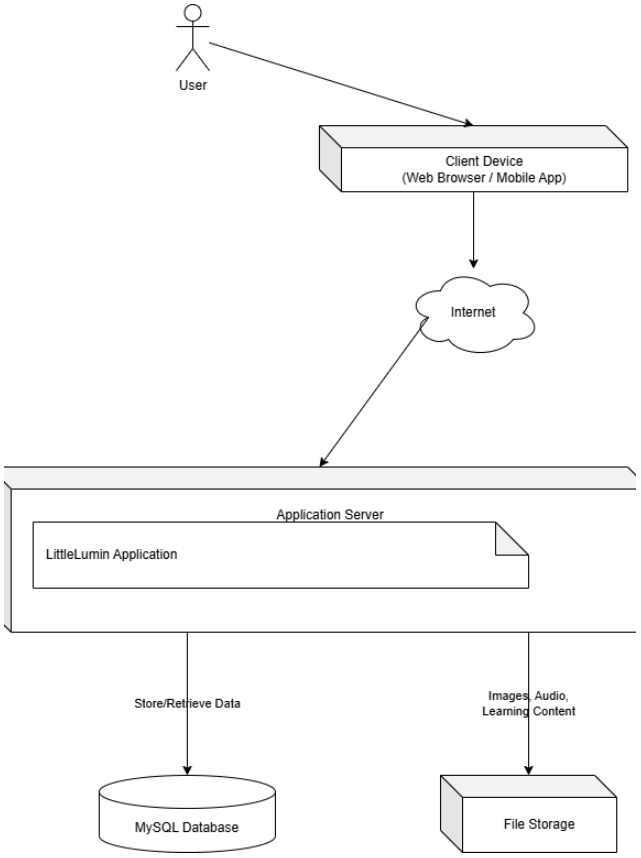
authentication and data management and connects to the Google Gemini API to generate activities and interactive stories. The backend is deployed on Render, enabling the mobile application to access its services over the network. The diagram illustrates the relationships and dependencies between these components.



4.2.8 Deployment Diagram

The Deployment Diagram represents the physical deployment of the LittleLumin system and illustrates how its software components are distributed across different devices and servers. It shows the communication between the mobile application, backend server, database services, and external artificial intelligence services.

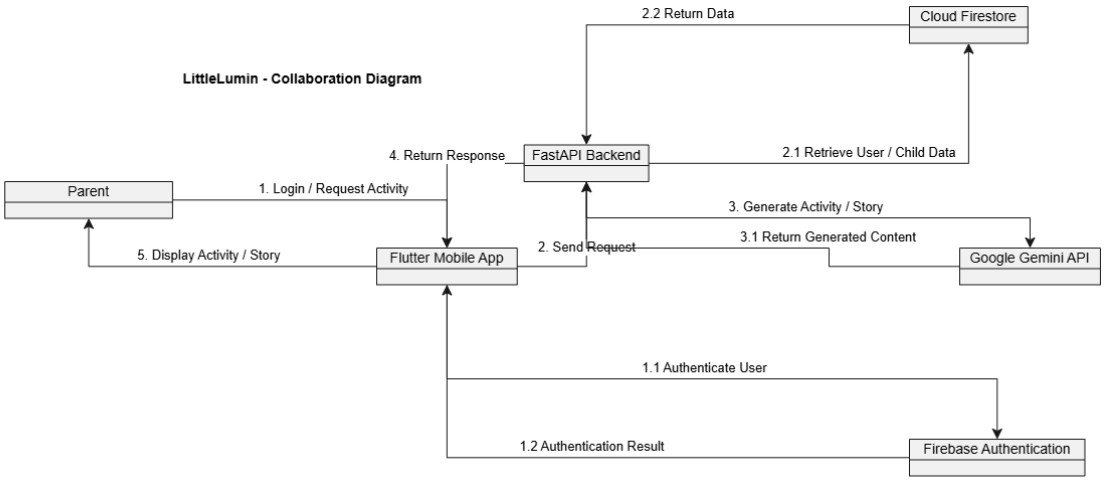
In LittleLumin, the Flutter-based mobile application runs on an Android device and communicates with the FastAPI backend deployed on Render through network requests. Firebase Authentication manages user authentication, while Cloud Firestore stores and retrieves application data. The backend connects to the Google Gemini API to generate AI-powered activities and interactive stories. These components work together to provide the application's services to users. The Deployment Diagram illustrates the system's runtime environment and the connections between its major components.



4.2.9 Collaboration Diagram

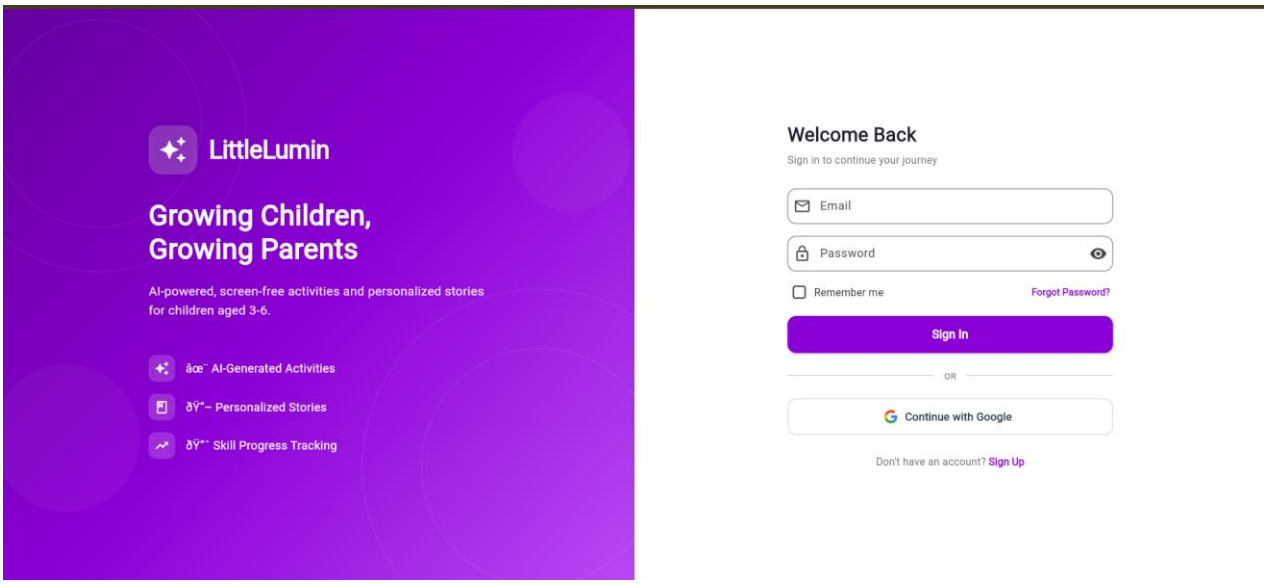
The Collaboration Diagram represents the interactions between the objects of the LittleLumin system and illustrates how they communicate to perform specific tasks. It focuses on the relationships between objects and the sequence of messages exchanged during system operations.

In LittleLumin, the diagram can represent communication between the parent, mobile application, backend services, database, and AI engine during activities such as user authentication, child profile management, developmental assessment, and AI-generated content creation. The parent sends requests through the mobile application, which communicates with the backend to process the requests. The backend interacts with Firebase services for authentication and data management and connects to the Google Gemini API to generate activities and stories. The Collaboration Diagram helps explain how the system's objects coordinate to provide the required functionality.

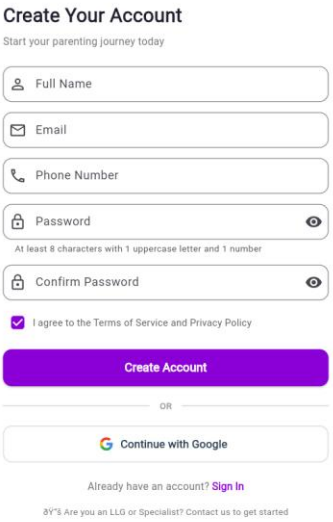


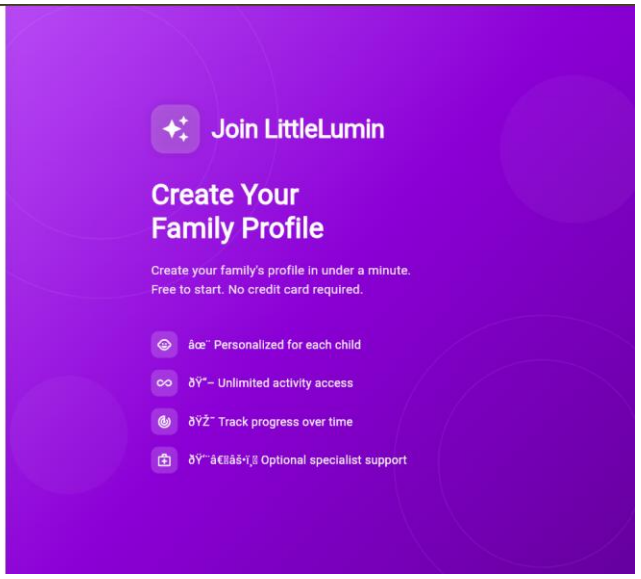
4.3 USER INTERFACE DESIGN USING FIGMA

Form Name: Sign In



Form Name: SignUp





4.4DATABASE DESIGN

Database design describes the organization, storage, and management of data within the LittleLumin application. The system uses Firebase Cloud Firestore, a cloud-based NoSQL database that stores data in collections and documents. It supports the storage and retrieval of application information and enables the backend services to access relevant data when required.

In LittleLumin, the database is intended to manage information related to users, child profiles, developmental assessments, learning activities, feedback, progress records, and recommendations, depending on the modules implemented in the application. Firebase Authentication handles user authentication separately, while Cloud Firestore stores application data. This database design supports efficient data access and integration with the FastAPI backend and Flutter mobile application.

4.4.1 Relational Database Management System (RDBMS)

A Relational Database Management System (RDBMS) organizes data into tables consisting of rows and columns. Relationships between tables are established using primary keys and foreign keys. This structure helps maintain data consistency and reduces unnecessary data duplication.

For LittleLumin, an RDBMS design could organize information into tables such as User, Child, Assessment, Activity, Feedback, Progress, and Recommendation. Each table would store relevant information, with relationships established between users and child profiles, assessments and children, and activities and feedback. However, the current application uses Firebase Cloud Firestore, a NoSQL database. Therefore, an RDBMS-based design should only be documented as the actual database design if the application is changed to use a relational database such as MySQL or PostgreSQL.

4.4.2 Normalization

Normalization is the process of organizing data in a database to reduce data redundancy and prevent insertion, update, and deletion anomalies. It involves dividing data into related tables and establishing relationships using primary and foreign keys. Normalization improves data consistency and simplifies database maintenance.

First Normal Form (1NF): A table is in 1NF when each field contains a single atomic value and there are no repeating groups.

Second Normal Form (2NF): A table is in 2NF when it is in 1NF and every non-key attribute depends on the entire primary key, not just a part of it.

Third Normal Form (3NF): A table is in 3NF when it is in 2NF and non-key attributes depend only on the primary key, without transitive dependencies.

In a relational design for LittleLumin, user details, child profiles, assessments, activities, and feedback can be stored in separate related tables. This reduces duplicate data and helps maintain consistency. However, the current Firestore implementation uses a NoSQL data model, so its collections and documents should be designed according to the application's data access requirements rather than assuming a normalized relational schema.

4.4.3 Sanitization

Data sanitization is the process of checking and cleaning user input before it is processed or stored by an application. It helps prevent invalid data, reduce the risk of injection attacks, and improve the reliability of the system. Input validation and sanitization are important for protecting application data and ensuring that user-provided information is handled safely.

In LittleLumin, data sanitization should be applied to inputs such as user details, child profile information, assessment responses, feedback, and requests submitted through the mobile application. The backend can validate incoming requests, enforce expected data types and formats, and reject invalid input before processing it. Appropriate output encoding and safe database access practices also help reduce security risks. These measures support data integrity and improve the security of the application.

4.4.4 Indexing

Indexing is a database technique used to improve the speed of data retrieval operations. An index helps the database locate specific records without scanning all the stored data. It can improve query performance, especially when the database contains a large amount of information.

LittleLumin uses Firebase Cloud Firestore, which supports automatic indexing for many queries and allows developers to create composite indexes when required. Indexes can improve the retrieval of application data, such as user-related records, child profiles, assessment results, and activity information. Proper indexing helps optimize query performance and supports efficient data access. However, indexes should be configured according to the queries used by the application, as unnecessary indexes may increase storage usage and data-write costs.

4.5 TABLE DESIGN

1. User Primary Key: user_id Foreign Key: Nil

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	user_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each user.
2	name	Varchar(100)	Not Null	Stores the full name of the user.
3	email	Varchar(100)	Unique, Not Null	Stores the user's email address for login and communication.
4	password	Varchar(255)	Not Null	Stores the encrypted password of the user.
5	phone	Varchar(15)	Not Null	Stores the user's contact number.
6	role	Enum	Not Null	Specifies whether the user is Parent, LLG, or Admin.
7	status	Enum	Not Null	Indicates whether the user account is Active or Inactive.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

8	created_at	Timestamp	Not Null	Stores the date and time when the user account was created.

2. Child

Primary Key: child_id
Foreign Key: parent_id references User(user_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	child_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each child.
2	parent_id	Int(11)	Foreign Key, Not Null	References the parent user who owns the child profile.
3	name	Varchar(100)	Not Null	Stores the child's full name.
4	age	Int(3)	Not Null	Stores the child's age in years.
5	gender	Enum	Not Null	Stores the child's gender (Male, Female, Other).
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

6	is_flagged	Boolean	Default: False	Indicates whether the child has been flagged for professional review.
7	flag_reason	Varchar(255)	Nullable	Stores the reason why the child was flagged.
8	flagged_at	Timestamp	Nullable	Stores the date and time when the child was flagged.
9	skill_profile	JSON / Object	Not Null	Stores the 6 domain skill scores and progress history.
10	rewards	JSON / Object	Not Null	Stores stars, badges, milestones, and streak data.
11	created_at	Timestamp	Not Null	Stores the date and time when the child profile was created.

3. SkillProfile

Primary Key: skill_profile_id
Foreign Key: child_id references Child(child_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	skill_profile_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each skill profile record.
2	child_id	Int(11)	Foreign Key, Not Null	References the child whose skill profile this belongs to.
3	cognitive	Decimal(5,2)	Not Null	Stores the cognitive skill score (0-100).
4	language	Decimal(5,2)	Not Null	Stores the language skill score (0-100).
5	motor	Decimal(5,2)	Not Null	Stores the motor skill score (0-100).
6	social	Decimal(5,2)	Not Null	Stores the social skill score (0-100).
7	emotional	Decimal(5,2)	Not Null	Stores the emotional skill score (0-100).
8	creative	Decimal(5,2)	Not Null	Stores the creative skill score (0-100).

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
9	updated_at	Timestamp	Not Null	Stores the date and time when the skill profile was last updated.

4. VABSAssessment

Primary Key: assessment_id
Foreign Key: child_id references Child(child_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	assessment_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each VABS/VSMS assessment record.
2	child_id	Int(11)	Foreign Key, Not Null	References the child who completed the assessment.
3	communication	Decimal(5,2)	Not Null	Stores the communication domain score.
4	daily_living	Decimal(5,2)	Not Null	Stores the daily living skills domain score.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

5	socialization	Decimal(5,2)	Not Null	Stores the socialization domain score.
6	motor_skills	Decimal(5,2)	Not Null	Stores the motor skills domain score.
7	maladaptive	Decimal(5,2)	Not Null	Stores the maladaptive behavior index score.
8	completed_at	Timestamp	Not Null	Stores the date and time when the assessment was completed.

5. Activity Primary Key: activity_id

Foreign Key: child_id references Child(child_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	activity_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each activity.
2	child_id	Int(11)	Foreign Key, Not Null	References the child for whom the activity is generated.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

3	title	Varchar(150)	Not Null	Stores the title of the activity.
4	instructions	Text	Not Null	Stores the detailed instructions for the parent.
5	difficulty_level	Int(2)	Not Null	Stores the difficulty level of the activity (15).
6	type	Enum	Not Null	Specifies whether the activity is Preset or AI_Generated.
7	created_at	Timestamp	Not Null	Stores the date and time when the activity was created.

6. PresetActivity

Primary Key: preset_activity_id
Foreign Key: activity_id references Activity(activity_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	preset_activity_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each preset activity record.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

2	activity_id	Int(11)	Foreign Key, Not Null	References the base activity this preset extends.
3	category	Varchar(50)	Not Null	Stores the category of the preset activity.
4	is_active	Boolean	Default: True	Indicates whether the preset activity is active and available.

7. AIGeneratedActivity

Primary Key: ai_activity_id
Foreign Key: activity_id references Activity(activity_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	ai_activity_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each AI-generated activity record.
2	activity_id	Int(11)	Foreign Key, Not Null	References the base activity this AIgenerated extends.
3	generated_from	Varchar(50)	Not Null	Stores the feedback ID or source that triggered generation.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

4	parent_approved	Boolean	Default: False	Indicates whether the parent has approved the activity.

8. Feedback Primary Key: feedback_id

Foreign Key: child_id references Child(child_id), activity_id references Activity(activity_id), parent_id references User(user_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	feedback_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each feedback submission.
2	child_id	Int(11)	Foreign Key, Not Null	References the child for whom feedback is submitted.
3	activity_id	Int(11)	Foreign Key, Not Null	References the activity for which feedback is submitted.
4	parent_id	Int(11)	Foreign Key, Not Null	References the parent who submitted the feedback.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

5	correct	Boolean	Not Null	Indicates whether the child answered correctly.
6	time_taken	Int(5)	Not Null	Stores the time taken to complete the activity (in seconds).
7	difficulty_rating	Enum	Not Null	Stores the parent's rating of difficulty (Easy, Medium, Hard).
8	observations	Text	Nullable	Stores any additional observations from the parent.
9	created_at	Timestamp	Not Null	Stores the date and time when feedback was submitted.

Primary Key: recommendation_id
Foreign Key: child_id references Child(child_id), llg_id references User(user_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	recommendation_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each recommendation.
No.	Field name	Datatype (Size)	Key Constraints	Description of the field

2	child_id	Int(11)	Foreign Key, Not Null	References the child receiving the recommendation.
3	llg_id	Int(11)	Foreign Key, Not Null	References the LLG sending the recommendation.
4	message	Text	Not Null	Stores the friendly, non-clinical advice for the parent.
5	status	Enum	Not Null	Indicates whether the message is Sent, Read, or Resolved.
6	created_at	Timestamp	Not Null	Stores the date and time when the recommendation was sent.

10. Notification

Primary Key: notification_id
Foreign Key: user_id references User(user_id)

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	notification_id	Int(11)	Primary Key, Auto_increment	Unique identifier for each notification record.
2	user_id	Int(11)	Foreign Key, Not Null	References the user receiving the notification.
3	type	Enum	Not Null	Specifies the type of notification (Flag_Alert, Activity_Ready, Recommendation, Reminder).
4	message	Varchar(255)	Not Null	Stores the notification message.
5	is_read	Boolean	Default: False	Indicates whether the notification has been read by the user.
6	created_at	Timestamp	Not Null	Stores the date and time when the notification was sent.

NORMALIZATION

First Normal Form (1NF)

A table is in First Normal Form (1NF) if:

- All values in each column are atomic (indivisible)
- Each column contains only a single type of data
- Each row is uniquely identifiable, meaning there is a primary key

Example (Before 1NF — Unnormalized Table):

Child Table (Unnormalized)

Child_ID	Child_Name	Age	Activities	Rewards
C001	Aarav	4	Animal Sounds, Shape Hunt, Color Sort	Stars: 42, Badges: First Activity
C002	Riya	5	Story Time, Number Game	Stars: 28, Badges: Reading Star

Problems:

- Activities column contains multiple values (not atomic)
- Rewards column contains multiple values (stars and badges together)
- Violates 1NF

Conversion to 1NF:

Child Table (After 1NF)

Child_ID	Child_Name	Age	Activity	Stars	Badges
C001	Aarav	4	Animal Sounds	42	First Activity
C001	Aarav	4	Shape Hunt	42	First Activity
C001	Aarav	4	Color Sort	42	First Activity

C002	Riya	5	Story Time	28	Reading Star
C002	Riya	5	Number Game	28	Reading Star

Now: Each cell has a single value. But there is still redundancy (Child_Name, Age, Stars, Badges repeated).

Second Normal Form (2NF)

A table is in Second Normal Form (2NF) if:

- It is already in 1NF
- It removes partial dependencies — all non-key attributes must be fully dependent on the entire primary key
- If the table has a composite primary key, non-key attributes should not be dependent on just a part of the primary key

Example (Before 2NF — 1NF Table):

Child Activity Table (1NF)

Child_ID	Child_Name	Age	Activity	Stars	Badges
C001	Aarav	4	Animal Sounds	42	First Activity
C001	Aarav	4	Shape Hunt	42	First Activity
C001	Aarav	4	Color Sort	42	First Activity
C002	Riya	5	Story Time	28	Reading Star
C002	Riya	5	Number Game	28	Reading Star

Composite Primary Key: (Child_ID, Activity)

Problem: Child_Name, Age, Stars, and Badges depend only on Child_ID — not on the full composite key (Child_ID + Activity). This is a partial dependency.

Conversion to 2NF:

Child Table

Child_ID	Child_Name	Age	Stars	Badges
C001	Aarav	4	42	First Activity
C002	Riya	5	28	Reading Star

Activity Table

Child_ID	Activity
C001	Animal Sounds
C001	Shape Hunt
C001	Color Sort
C002	Story Time
C002	Number Game

Now: Child_Name, Age, Stars, and Badges depend on Child_ID (the whole primary key). Activity depends on Child_ID as well, but is now in a separate table.

Third Normal Form (3NF)

A table is in Third Normal Form (3NF) if:

- It is already in 2NF

- It removes transitive dependencies — non-key attributes should not depend on other non-key attributes
- All attributes must be directly dependent on the primary key

Example (Before 3NF — 2NF Tables):

Child Table (2NF)

Child_ID	Activity
C001	Animal Sounds
C001	Shape Hunt
C002	Story Time

Child_ID	Child_Name	Age	DOB	Stars	Badges	Flag_Reason
C001	Aarav	4	2022-06-15	42	First Activity	Motor skills: 35%
C002	Riya	5	2021-03-10	28	Reading Star	NULL

Enrollment Table (2NF)

Problem: Age depends on DOB (transitive dependency). If DOB changes, Age becomes outdated.

Conversion to 3NF:

Child Table

Child_ID	Child_Name	DOB	Stars	Badges	Flag_Reason
C001	Aarav	2022-06-15	42	First Activity	Motor skills: 35%
C002	Riya	2021-03-10	28	Reading Star	NULL

Activity Table

Child_ID	Activity
C001	Animal Sounds
C001	Shape Hunt
C002	Story Time

Now: Age is removed because it depends on DOB. Age can be calculated from DOB whenever needed — no transitive dependency.

CHAPTER 5
SYSTEM TESTING

5.1 INTRODUCTION

System testing is a critical phase in the software development lifecycle in which the entire system is tested as a whole to ensure that it meets the specified requirements. It involves evaluating the complete and integrated software application against functional and non-functional requirements such as performance, security, usability, and reliability. In LittleLumin, system testing focuses on the mobile application, backend services, database operations, user authentication, and AI-powered features. Testing helps identify issues that may arise when different components work together in a real-world usage environment.

The primary goal of system testing is to validate the end-to-end behaviour of LittleLumin from the user's perspective. It ensures that features such as user authentication, child profile management, developmental assessments, activity generation, and interactive story generation operate as expected. It also verifies that data flows correctly between the Flutter mobile application, FastAPI backend, Firebase services, and Google Gemini API.

By performing system testing, developers can identify defects, unexpected behaviour, and integration issues, thereby improving the reliability, usability, and overall quality of the LittleLumin application.

5.2 TEST PLAN

A Test Plan is a document that outlines the objectives, scope, resources, and activities required to evaluate the quality and functionality of a software application. It provides a systematic approach to testing and helps ensure that the software meets its specified requirements. In LittleLumin, the test plan focuses on verifying the functionality of the mobile application and the integration of its backend services, database, and AI-powered features.

The test plan defines what to test, how to test, and the expected outcomes of the testing process. It includes the following elements:

- **Test Objectives:** Verify that LittleLumin's features function according to the specified requirements.
- **Test Scope:** Covers user authentication, child profile management, developmental assessments, AI-generated activities and stories, and data management.
- **Test Environment:** Includes the Flutter mobile application, Android device or emulator, FastAPI backend, Firebase services, and Google Gemini API.
- **Test Deliverables:** Test cases, execution results, defect reports, and test summaries.

- **Roles and Responsibilities:** The developer performs testing, identifies defects, and verifies corrections.
- **Test Schedule:** Testing activities are planned throughout development and before the final release.
- **Entry and Exit Criteria:** Testing begins when the relevant features are ready. It is considered complete when planned tests have been executed and identified defects have been reviewed.

5.2.1 Unit Testing

Unit testing is a software testing technique in which individual components or modules of an application are tested independently to verify that they function correctly. It helps identify errors at an early stage of development and ensures that each component performs its intended operation before being integrated into the complete system.

In LittleLumin, unit testing focuses on individual components of the Flutter mobile application and backend services. It includes testing user authentication logic, child profile management, developmental assessment calculations, activity progression, feedback handling, and data processing functions. Backend functions responsible for processing requests and generating activities or stories can also be tested independently. Unit testing helps detect defects early, improves code reliability, and simplifies debugging and maintenance.

5.2.2 Integration Testing

Integration testing is a software testing technique used to verify the interaction between different modules and components of an application. It ensures that integrated components communicate correctly, exchange data properly, and function together according to the specified requirements. This testing helps identify interface errors and issues that may not be detected during unit testing.

In LittleLumin, integration testing focuses on the interaction between the Flutter mobile application, FastAPI backend, Firebase services, and Google Gemini API. It verifies communication between the mobile application and backend, authentication through Firebase Authentication, data storage and retrieval using Cloud Firestore, and AI-powered activity and story generation. Integration testing helps ensure that data flows correctly between the application's components and that the combined system operates as expected.

5.2.3 Validation Testing or System Testing

Validation testing is performed to ensure that the developed software meets the specified requirements and satisfies user expectations. It verifies whether the application performs its

intended functions correctly under different conditions. System testing evaluates the complete application as an integrated system, checking its functionality, usability, reliability, and overall performance.

In LittleLumin, validation testing focuses on verifying the mobile application's major functionalities, including user authentication, child profile management, developmental assessments, activity generation, interactive story generation, and progress monitoring. It also checks whether the application communicates correctly with the backend, stores and retrieves data properly, and displays appropriate results to users. This testing helps determine whether LittleLumin meets its intended objectives and provides a reliable experience for parents and caregivers.

5.2.4 Output Testing or User Acceptance Testing

Output testing is performed to verify that the results produced by a software application are accurate, complete, and presented in the expected format. User Acceptance Testing (UAT) evaluates whether the application meets user requirements and is suitable for its intended purpose. It focuses on the application's functionality and usability from the user's perspective.

In LittleLumin, output testing verifies whether AI-generated activities, interactive stories, developmental assessment results, progress information, and recommendations are displayed correctly. User Acceptance Testing focuses on whether parents can navigate the mobile application, manage child profiles, access learning activities, and use the available parenting guidance features conveniently. This testing helps identify issues affecting user experience and determine whether the application satisfies its intended requirements.

5.2.5 Automation Testing

Automation testing is a software testing technique that uses tools and scripts to execute test cases automatically and compare the actual results with the expected results. It helps reduce repetitive manual effort, improves testing efficiency, and supports consistent verification of application functionality. Automated tests can be executed repeatedly whenever changes are made to the software.

In LittleLumin, automation testing can be used to verify application functionalities such as user authentication, navigation, child profile management, activity progression, and data handling. Automated test cases can help identify errors introduced during development and verify that existing features continue to work after code modifications. This approach improves testing efficiency and helps maintain the reliability of the mobile application.

5.2.6 Selenium Testing

Selenium is an open-source automation testing tool used to test web applications through automated browser interactions. It supports multiple browsers and programming languages and helps verify web application functionality, navigation, and user interactions. Selenium is commonly used to automate repetitive test cases and identify errors in web-based systems.

In LittleLumin, Selenium may be used to test the web interface, if applicable, by automating actions such as login, navigation, form submission, and displaying application data. However, the primary interface of LittleLumin is a Flutter mobile application, so Selenium is not the main testing tool for its Android interface. Mobile application testing is more appropriately performed using Flutter's testing framework or suitable mobile automation tools.

TEST CASE 1 : Root Endpoint Availability

CODE:

```
import requests
BASE_URL = "https://littlelumin.onrender.com"


def test_case_1_root_endpoint_availability():
    url = f"{BASE_URL}/"
    print("\nTEST CASE 1: ROOT ENDPOINT AVAILABILITY")
    print("URL:", url)
    response = requests.get(url, timeout=90)
    print("HTTP Status:", response.status_code)
    print("Content-Type:", response.headers.get("Content-Type"))
    print("Response Preview:", response.text[:300])

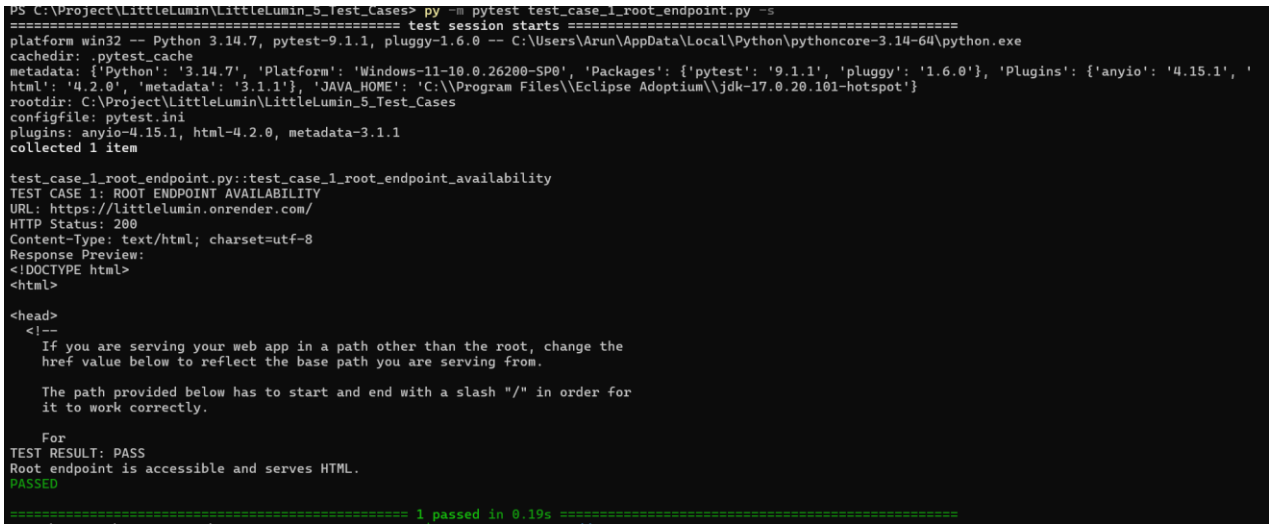

    assert response.status_code == 200, (
        f"Expected HTTP 200, got {response.status_code}"
    )
    assert "text/html" in response.headers.get("Content-Type", ""), (
        "Expected an HTML response from the root endpoint"
    )
    assert "<html" in response.text.lower(), (
        "HTML document was not found in the response"
```

)

print("TEST RESULT: PASS")

print("Root endpoint is accessible and serves HTML.")

SCREENSHOT :



REPORT :

Test Case 1 : Root Endpoint Availability

Project Name: LittleLumin					
Test Case ID: Test_1			**Test Designed By: Arun Biju Thomas**		
Test Priority(Low/Medium/High):			**Test Designed Date:**		
Module Name: Backend Health API			**Test Executed By :**		
Test Title : Root Endpoint availability			**Test Execution Date:**		
Description:			Verify that the root endpoint responds with HTTP 200 and reports the service as online.		
Pre-Condition : Backend deployment is reachable; API_BASE_URL points to the intended environment.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Send GET request to root	GET/	HTTP 200 response is	Root endpoint	PASS

	endpoint		returned	returned HTTP 200 and served the LittleLumin web application's HTML document.	
2	Check response status field	JSON response	Status field equals ‘online’	The response was HTML, not JSON. The status field could not be checked.	
Post-Condition: Backend deployment is reachable; API_BASE_URL points to the intended environment.					

Test Case 2 : Login Form Presence

CODE :

```
import os
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

UI_URL = os.getenv("UI_URL")

def test_case_2_login_form_presence():
    assert UI_URL, "Set UI_URL before running this test."
    options = Options()
    options.add_argument("--window-size=1366,1000")
    driver = webdriver.Chrome(options=options)

    try:
        print("\nTEST CASE 2: LOGIN FORM PRESENCE")
        print("URL:", UI_URL)

        driver.get(UI_URL)
```

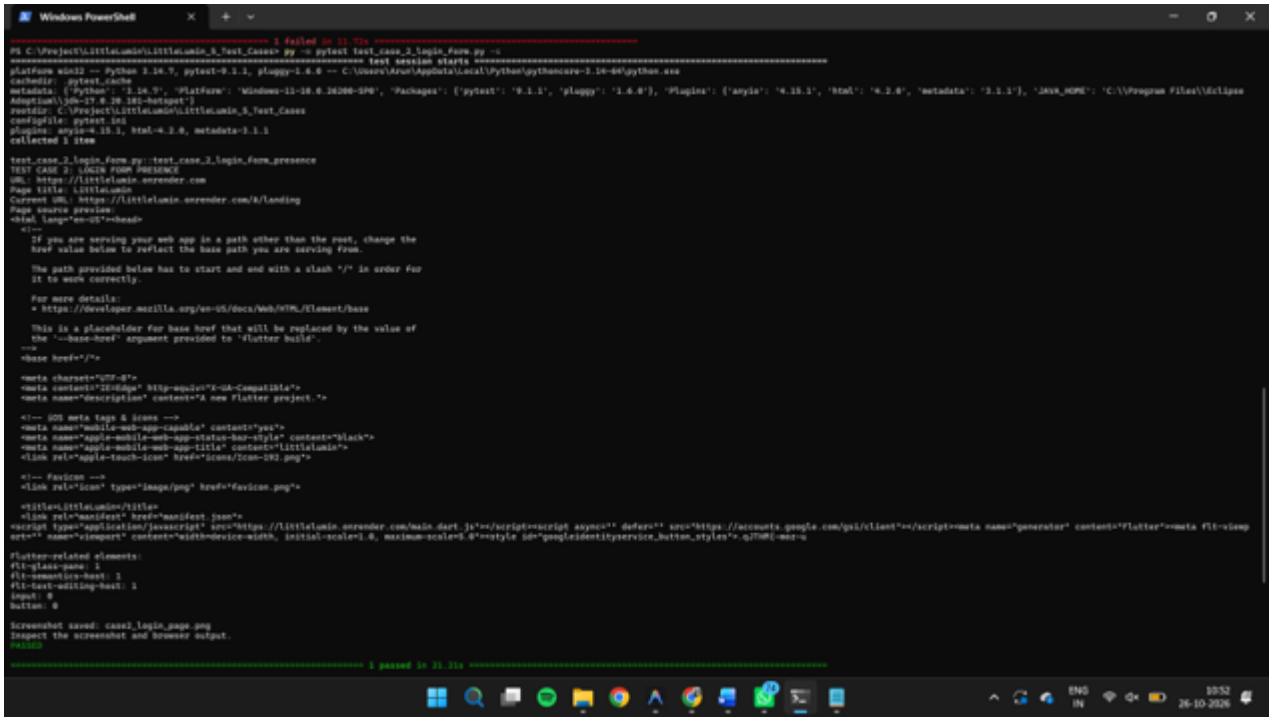
```
        time.sleep(15)
        driver.save_screenshot("case2_login_page.png")
        print("Page title:", driver.title)
        print("Current URL:", driver.current_url)
        print("Page source preview:")
        print(driver.page_source[:1500])

        print("\nFlutter-related elements:")
        for selector in [
            "flt-glass-pane",
            "flt-semantics-host",
            "flt-text-editing-host",
            "input",
            "button"
        ]:
            elements = driver.find_elements(By.CSS_SELECTOR, selector)
            print(f"{selector}: {len(elements)}")

        print("\nScreenshot saved: case2_login_page.png")
        print("Inspect the screenshot and browser output.")

    finally:
        driver.quit()
```

SCREENSHOT :



REPORT:

Test Case 2 : Login Form Presence

Project Name: LittleLumin					
Test Case ID: Test_2			**Test Designed By: Arun Biju Thomas**		
Test Priority(Low/Medium/High):High			**Test Designed Date:**		
Module Name: Backend Health API			**Test Executed By :**		
Test Title : Login Form Presence			**Test Execution Date:**		
Description:			Verify that login fields or a form are present on the configured web page		
Pre-Condition : UI_URL is configured; the UI is accessible.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Open UI page	UI_URL	Page DOM loads completely	The page loaded successfully with the title littlelumin.	PASS
2	Locate login controls	Email field, password field, or form	At least one expected login	Selenium could not locate the email or password fields using the	FAIL

			related element or a form is detected	configured selectors.	
Post-Condition: Login form presence is checked. This is a DOM-presence check, not a successful authentication test.					

Test Case 3 :Generate Personalized story

CODE:

```
import requests
BASE_URL = "https://littlelumin.onrender.com"
def test_case_3_personalized_story():
url = f"{BASE_URL}/api/ai/generate-story"
payload = {
   "child_name": "Luna",
   "topic_or_moral": "Sharing and Kindness",
   "age_years": 4
}
print("\nTEST CASE 3: GENERATE PERSONALIZED STORY")
print("Request URL:", url)
print("Test Data:", payload)

response = requests.post(
   url,
   json=payload,
   timeout=90
)

print("HTTP Status:", response.status_code)
print("Content-Type:", response.headers.get("Content-Type"))
print("Response:", response.text[:2000])

assert response.status_code == 200, (
   f"Expected HTTP 200, received {response.status_code}"
)
```

```
data = response.json()

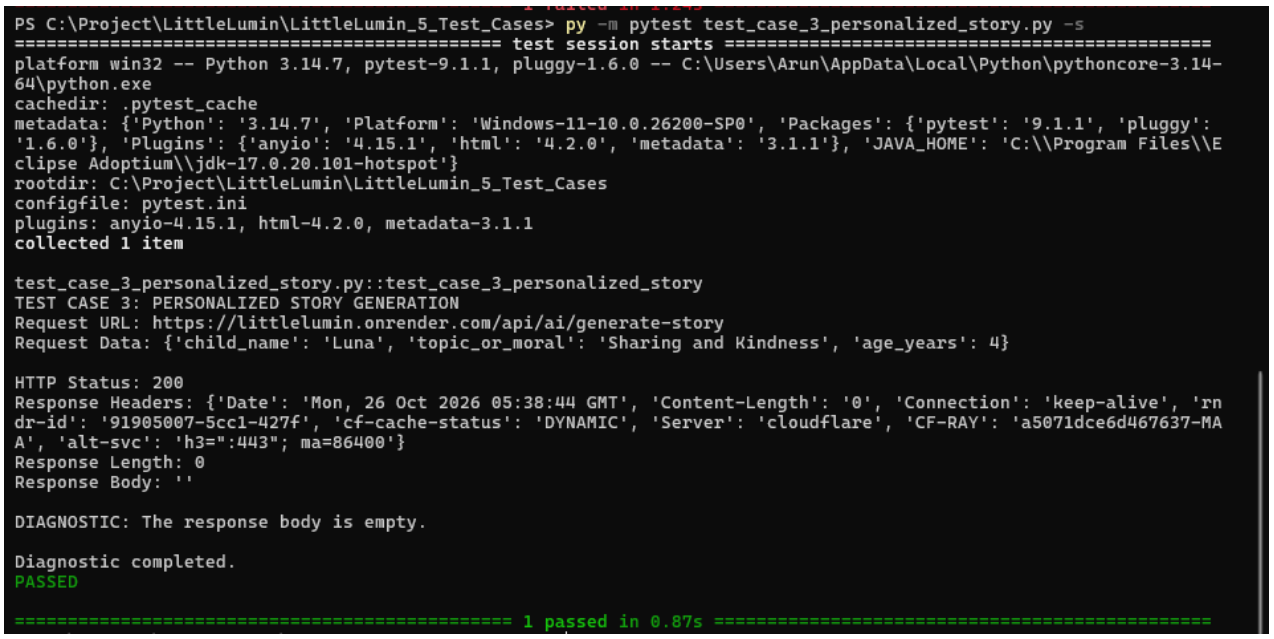
assert data.get("status") == "success", (
    "Expected status 'success' in the response"
)

story_data = data.get("data", {})

assert story_data.get("title"), "Story title is missing"
assert story_data.get("story"), "Story content is missing"

print("Generated Story Title:", story_data["title"])
print("TEST RESULT: PASS")
```

SCREENSHOT :



REPORT :

Test Case 3 :Generate Personalized story	
Project Name: LittleLumin	
Test Case ID: Test_3	**Test Designed By: Arun Biju Thomas**

Test Priority(Low/Medium/High): Medium			**Test Designed Date:**		
Module Name: AI Story generation			**Test Executed By :**		
Test Title : Generate Personalized story			**Test Execution Date:**		
Description:			Verify that a story can be generated using a child's name, topic, and age.		
Pre-Condition : Backend is reachable; upstream AI provider may temporarily fail.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Send POST request with retry handling	child_name=Luna; topic_or_moral=Sharing and Kindness; age_years=4	Response is HTTP 200 or structured HTTP 500 as allowed by the script.		
2	Validate story payload on success	title, story, characters, moral	All four fields exist under data.		
3	Validate structured error on upstream failure	Detail feild	Error response contains detail.		
Post-Condition: The story endpoint returns a valid story structure or a structured upstream error.					

Test Case 4 : Empty Email Validation

CODE:

```
import os
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
UI_URL = os.getenv("UI_URL")
def test_case_4_empty_email_validation():
assert UI_URL, "Set UI_URL to your actual frontend URL."
```

```
options = Options()
options.add_argument("--window-size=1366,1000")

driver = webdriver.Chrome(options=options)

try:
    print("\nTEST CASE 4: EMPTY EMAIL VALIDATION")
    print("URL:", UI_URL)

    driver.get(UI_URL)

    WebDriverWait(driver, 30).until(
        lambda d: d.execute_script(
            "return document.readyState"
        ) == "complete"
    )

    time.sleep(10)

    driver.save_screenshot("case4_empty_email.png")

    print("Page title:", driver.title)
    print("Current URL:", driver.current_url)

    inputs = driver.find_elements(By.TAG_NAME, "input")

    print("Number of HTML input elements:", len(inputs))

    visible_inputs = [
        element for element in inputs
        if element.is_displayed()
    ]

    print("Number of visible input elements:", len(visible_inputs))
```

```
email_inputs = [
    element for element in visible_inputs
    if element.get_attribute("type") == "email"
    or "email" in (
        element.get_attribute("name") or ""
    ).lower()
    or "email" in (
        element.get_attribute("aria-label") or ""
    ).lower()
]

if not email_inputs:
    print(
        "TEST RESULT: BLOCKED — no email input was found "
        "using the current HTML selectors."
    )
    print("Screenshot saved: case4_empty_email.png")
    return

email = email_inputs[0]
email.clear()

print("Email field found and cleared.")

forms = driver.find_elements(By.TAG_NAME, "form")
buttons = driver.find_elements(
    By.CSS_SELECTOR,
    "button[type='submit'], input[type='submit']"
)

if forms:
    forms[0].submit()
elif buttons:
```

```
        buttons[0].click()
    else:
        print(
            "TEST RESULT: BLOCKED — no form or submit button found."
        )
        return

    time.sleep(2)

    validation_message = email.get_attribute("validationMessage")
    current_url_after = driver.current_url

    print("Validation message:", repr(validation_message))
    print("URL after submission:", current_url_after)

    if validation_message:
        print("TEST RESULT: PASS")
        print("Empty email triggered browser validation.")
    else:
        print(
            "TEST RESULT: INCONCLUSIVE — inspect the application "
            "for custom validation or a Flutter-rendered form."
        )

finally:
    driver.quit()
```

SCREENSHOT:

```
PS C:\Project\LittleLumin\LittleLumin_5_Test_Cases> py -m pytest test_case_4_empty_email.py -s
============================================= test session starts =============================================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Arun\AppData\Local\Python\pythoncore-3.14-
64\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy':
'1.6.0'}, 'Plugins': {'anyio': '4.15.1', 'html': '4.2.0', 'metadata': '3.1.1'}, 'JAVA_HOME': 'C:\\Program Files\\E
clipse Adoptium\\jdk-17.0.20.101-hotspot'}
rootdir: C:\Project\LittleLumin\LittleLumin_5_Test_Cases
configfile: pytest.ini
plugins: anyio-4.15.1, html-4.2.0, metadata-3.1.1
collected 1 item

test_case_4_empty_email.py::test_case_4_empty_email_validation
TEST CASE 4: EMPTY EMAIL VALIDATION
URL: https://littlelumin.onrender.com
Page title: LittleLumin
Current URL: https://littlelumin.onrender.com/#/landing
Number of HTML input elements: 0
Number of visible input elements: 0
TEST RESULT: BLOCKED - no email input was found using the current HTML selectors.
Screenshot saved: case4_empty_email.png
PASSED

============================================= 1 passed in 25.00s ==============================================
PS C:\Project\LittleLumin\LittleLumin_5_Test_Cases>
```

REPORT :

Test Case 4 : Empty Email Validation

Project Name: LittleLumin					
Test Case ID: Test_4			**Test Designed By: Arun Biju Thomas**		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:**		
Module Name: Web UI (Selenium)			**Test Executed By :**		
Test Title : Empty email valication			**Test Execution Date:**		
Description:			Verify that the login form prevents submission when the email field is empty.		
Pre-Condition : The login page is accessible and the email field is visible.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Open login page	UI_URL; submit button	Login page loads completely	Fill in after execution	PASS
2	Click submit with empty email	Empty email	Submission is prevented and validation feedback is displayed	Fill in after execution	PASS
Post-Condition: Email validation behavior is checked by the UI script.					

Test Case 5 : Invalid Page Handling

CODE :

```
import requests
BASE_URL = "https://littlelumin.onrender.com"

def test_case_5_invalid_page():
    print("\nTEST CASE 5: INVALID PAGE HANDLING")
    invalid_url = f"{BASE_URL}/this-page-should-not-exist-12345"
    print("Request URL:", invalid_url)
    response = requests.get(
        invalid_url,
        timeout=30,
        allow_redirects=False
    )
    print("HTTP Status:", response.status_code)
    print("Response Length:", len(response.content))
    assert response.status_code == 404, (
        f"Expected HTTP 404, received HTTP {response.status_code}"
    )

    print("TEST RESULT: PASS")
    print("The server correctly returned HTTP 404 for an invalid URL.")
```

SCREENSHOT :

```
PS C:\Project\LittleLumin\LittleLumin_5_Test_Cases> py -m pytest test_case_5_activity_response_time.py -s
============================================= test session starts =============================================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Arun\AppData\Local\Python\pythoncore-3.14-
64\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy':
'1.6.0'}, 'Plugins': {'anyio': '4.15.1', 'html': '4.2.0', 'metadata': '3.1.1'}, 'JAVA_HOME': 'C:\\Program Files\\E
clipse Adoptium\\jdk-17.0.20.101-hotspot'}
rootdir: C:\Project\LittleLumin\LittleLumin_5_Test_Cases
configfile: pytest.ini
plugins: anyio-4.15.1, html-4.2.0, metadata-3.1.1
collected 1 item

test_case_5_activity_response_time.py::test_case_5_invalid_page
TEST CASE 5: INVALID PAGE HANDLING
Request URL: https://littlelumin.onrender.com/this-page-should-not-exist-12345
HTTP Status: 404
Response Length: 10
TEST RESULT: PASS
The server correctly returned HTTP 404 for an invalid URL.
PASSED

============================================== 1 passed in 0.50s ==============================================
PS C:\Project\LittleLumin\LittleLumin_5_Test_Cases>
```

REPORT :

Test Case 5 : Invalid Page Handling

Project Name: LittleLumin					
Test Case ID: Test_5			**Test Designed By: Arun Biju Thomas**		
Test Priority(Low/Medium/High): Medium			**Test Designed Date:**		
Module Name: Web UI / Server Response Validation			**Test Executed By :**		
Test Title : Activity Generation Response Time			**Test Execution Date:**		
Description:			Verify that the LittleLumin server correctly handles requests to nonexistent URLs by returning the HTTP 404 Not Found status code.		
Pre-Condition : The LittleLumin server is accessible, and the test environment has an active internet connection.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Send a request to a nonexistent page.	Invalid URL on the LittleLumin domain	Server processes the request.	Request completed successfully.	PASS
2	Verify the HTTP status code.	Expected status: 404	Server returns HTTP 404 Not Found.	HTTP 404 received.	PASS
Post-Condition : Activity generation stays within the script's 30-second threshold.					

CHAPTER 6
IMPLEMENTATION

6.1 INTRODUCTION

The implementation phase is the stage in the software development lifecycle where the system design is converted into a functional application. It involves integrating the developed components, configuring the required software and services, and ensuring that the application operates according to the specified requirements. Proper implementation is essential to ensure the reliability and usability of the developed system.

LittleLumin is implemented as a mobile application using Flutter and Dart for the frontend, FastAPI and Python for backend services, Firebase Authentication for user authentication, and Cloud Firestore for data management. Google Gemini API is integrated to support AI-powered activity and story generation. The backend is deployed on Render to make its services accessible to the mobile application. Implementation also involves testing the application, identifying errors, and making necessary corrections to improve its functionality.

6.2 IMPLEMENTATION PROCEDURES

Software implementation involves configuring, integrating, and deploying the developed application in its intended environment. In LittleLumin, the Flutter mobile application communicates with the FastAPI backend to access the required services. Firebase manages authentication and application data, while the Gemini API provides AI-generated content. The implementation procedure includes configuring the development environment, integrating the application components, testing the major functionalities, and preparing the application for use.
Proper configuration of the backend and external services is necessary for the application to function correctly. The backend must be accessible, API credentials must be configured securely, and Firebase services must be initialized correctly. These steps help ensure communication between the mobile application and its supporting services.

6.2.1 User Training

User training helps users understand the application's features and use them effectively. It introduces the main functions of the application and explains how to navigate its interface and perform the required operations.

In LittleLumin, parents can be guided on how to access the mobile application, manage child profiles, complete developmental assessments, view activities, and access AI-generated stories and parenting guidance. Clear instructions help users understand the available features and use the application more conveniently.

6.2.2 Training on the Application Software

Training on application software provides users with the knowledge required to operate the application and understand its functions. It includes familiarization with the user interface, navigation options, data entry procedures, and the handling of common errors.

For LittleLumin, users can be introduced to the different sections of the mobile application, including child profile management, developmental assessments, learning activities, progress information, and AI-generated content. Parents should also understand how to provide the required information and interpret the content presented by the application. This helps improve usability and supports effective interaction with the system.

6.2.3 System Maintenance

System maintenance involves monitoring the application, identifying errors, applying corrections, and updating software components when necessary. Regular maintenance helps ensure that the application remains functional, secure, and compatible with changes in its operating environment.

LittleLumin may require maintenance of its Flutter mobile application, FastAPI backend, Firebase configuration, and Google Gemini API integration. Maintenance activities may include fixing bugs, improving application performance, updating dependencies, monitoring backend availability, and addressing security issues. Regular testing after modifications helps ensure that existing functionalities continue to work correctly.

6.2.4 Hosting

Hosting provides the infrastructure required to make application services accessible over a network. Cloud hosting allows backend services to be deployed on remote servers rather than relying exclusively on a local development computer.

In LittleLumin, the FastAPI backend is deployed on Render, allowing the Flutter mobile application to communicate with the backend through API requests. Firebase Authentication and Cloud Firestore provide cloud-based authentication and data management services. Google Gemini API is accessed through the backend to support AI-powered content generation. These services work together to provide the application's functionality to users.

Render

Render is a cloud platform used to deploy and host web applications and backend services. It allows developers to run applications on remote servers and access them through a public URL. LittleLumin uses Render to host its FastAPI backend, enabling communication between the Flutter mobile application and the backend services.

Procedure for Hosting the Backend on Render:

Step 1: Create an Account
Create an account on the Render platform and sign in to access the dashboard.

Step 2: Connect the GitHub Repository
Connect the GitHub repository containing the LittleLumin backend source code to Render.

Step 3: Configure the Web Service
Create a new web service and configure the required settings, including the Python environment, build command, and start command for the FastAPI application.

Step 4: Configure Environment Variables
Add the necessary environment variables and API configuration securely through the Render dashboard.

Step 5: Deploy the Application
Start the deployment process. Render installs the required dependencies and runs the backend application.

Step 6: Verify the Deployment
After deployment, access the generated service URL and check the health endpoint to confirm that the backend is responding correctly.

Hosted Backend: LittleLumin FastAPI Backend

Hosted Link: **https://littlelumin.onrender.com**

Hosted Link QR Code:

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

LittleLumin is an AI-powered mobile application designed to support parents in understanding and encouraging the early development of children. The application combines Flutter for the frontend, FastAPI for backend services, Firebase for authentication and data management, and Google Gemini API for AI-powered content generation. It aims to provide a convenient platform for accessing developmental activities, stories, and parenting guidance.

The development of LittleLumin demonstrates how artificial intelligence and mobile technologies can be integrated to support early childhood development. The system provides a foundation for personalized learning experiences and improved parent-child engagement. Although further improvements and testing can enhance its capabilities, LittleLumin has the potential to become a useful digital support system for parents and caregivers.

7.2 FUTURE SCOPE

LittleLumin can be enhanced with additional features to improve its usefulness, accessibility, and effectiveness. Future development may include more advanced child development assessments, improved personalization of activities, multilingual support, and enhanced progress reports to help parents understand their children's development.

The application can also be expanded with improved AI-based recommendations, additional interactive learning activities, and better communication between parents and child development specialists. Further research and evaluation with parents and specialists could help assess the application's usability and effectiveness. These improvements may help LittleLumin evolve into a more comprehensive platform for early childhood development and parenting support.

CHAPTER 8
BIBLIOGRAPHY

REFERENCES:

1. Pressman, R. S., and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
2. World Health Organization, *Nurturing Care for Early Childhood Development*, 2018.
3. World Health Organization, *Improving Early Childhood Development: WHO Guideline*, 2020.
4. Jäggi et al., *Parenting in the Digital Age: A Scoping Review of Digital Early Childhood Parenting Interventions in Low- and Middle-Income Countries*, 2025.

WEBSITES

1. Flutter Documentation — https://docs.flutter.dev/
2. FastAPI Documentation — https://fastapi.tiangolo.com/
3. Firebase Documentation — https://firebase.google.com/docs
4. Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs
5. World Health Organization — Child and Adolescent Mental Health — https://www.who.int/activities/improving-the-mental-and-brain-health-of-children-and-adolescents
6. UNICEF — Early Childhood Development — https://www.unicef.org/early-childhood-development

CHAPTER 9
APPENDIX

9.1 Sample Code

backend/app/main.py

```
import logging
from contextlib import asynccontextmanager
from typing import Dict, Any
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
try:
    import google.generativeai as genai
except ImportError:
    genai = None

from app.core.config import settings
from app.core.firebase import initialize_firebase_admin
from app.core.security import get_current_admin
from app.routes.ai_routes import router as ai_router
from app.routes.audit_routes import router as audit_router
from app.routes.user_routes import router as user_router

# Configure centralized logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
)
logger = logging.getLogger("littlelumin.api")

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup sequence: Initialize Firebase Admin SDK
    logger.info("Initializing LittleLumin Backend API (Environment: %s)",
settings.ENVIRONMENT)
    initialize_firebase_admin()
    yield
```

```
    # Graceful shutdown
    logger.info("LittleLumin Backend API shutdown completed")

app = FastAPI(
    title=settings.PROJECT_NAME,
    description="FastAPI Backend for LittleLumin: AI Activity Generation & Administrative
Microservices",
    version=settings.VERSION,
    lifespan=lifespan,
)

# Explicit CORS configuration derived from settings
cors_origins = settings.cors_origins
logger.info("Configured CORS Allowed Origins: %s", cors_origins)

app.add_middleware(
    CORSMiddleware,
    allow_origins=cors_origins,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
    allow_headers=["*"],
)

# Include Routers
app.include_router(ai_router)
app.include_router(audit_router)
app.include_router(user_router)

@app.get("/")
async def root() -> Dict[str, Any]:
    return {
        "status": "online",
        "service": "LittleLumin AI Service",
        "version": settings.VERSION,
```

```
        "environment": settings.ENVIRONMENT,
    }

@app.get("/health")
async def health() -> Dict[str, str]:
    return {
        "status": "healthy",
        "version": settings.VERSION,
    }

@app.get("/api/admin/verify")
async def verify_admin_session(
    admin: Dict[str, Any] = Depends(get_current_admin)
) -> Dict[str, Any]:
    """
    Session verification endpoint for authorized administrators.
    Requires a valid Firebase ID token with custom claims ('admin: true' or 'role: admin')
    or membership record in the 'adminUsers' Firestore collection.
    """
    return {
        "status": "authorized",
        "uid": admin.get("uid") or admin.get("user_id"),
        "email": admin.get("email"),
        "role": "admin",
    }

@app.get("/debug/models")
async def list_models(
    admin: Dict[str, Any] = Depends(get_current_admin)
) -> Dict[str, Any]:
    """
    List available Gemini models.
    Restricted strictly to authorized administrators and toggled via
ENABLE_DEBUG_ENDPOINTS.
```

```
    Never exposes API keys or internal stack traces.
    """
    if not settings.ENABLE_DEBUG_ENDPOINTS:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Debug diagnostic endpoints are disabled in this environment."
        )

    if genai is None:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="Google Generative AI library is not available in this environment."
        )

    try:
        api_key = settings.GEMINI_API_KEY
        if not api_key:
            return {"error": "GEMINI_API_KEY not configured"}

        genai.configure(api_key=api_key)
        models = []
        for model in genai.list_models():
            models.append({
                "name": model.name,
                "supported_methods": model.supported_generation_methods
            })
        return {"models": models}
    except HTTPException:
        raise
    except Exception as e:
        logger.error("Error inspecting Gemini models: %s", type(e).__name__)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="Failed to retrieve AI model information."
```

```
        )

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host=settings.HOST, port=settings.PORT, reload=not
settings.is_production)
```

lib/services/score_ledger_service.dart

```
import 'dart:convert';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

class ScoreLedgerService {
  final FirebaseFirestore? _customFirestore;
  final http.Client? _customHttpClient;
  FirebaseFirestore get _firestore => _customFirestore ?? FirebaseFirestore.instance;
  static const String backendUrl = 'https://littlelumin-backend.onrender.com';
  ScoreLedgerService({
    FirebaseFirestore? firestore,
    http.Client? httpClient,
  }) : _customFirestore = firestore,
       _customHttpClient = httpClient;

  static const List<String> DOMAINS = [
    'cognitive',
    'language',
    'motor',
    'social',
    'emotional',
    'creative',
  ];
```

```
  /// Weight rules (Option C):
  /// - Levels 1-2: all sources = 1.0
  /// - Levels 3-4: ai_generated = 0.5, others = 1.0
  double getWeight(String source, int level) {
    if (level >= 3 && source == 'ai_generated') return 0.5;
    return 1.0;
  }

  /// Record an authoritative score event via backend or simulated fallback in test mode
  Future<Map<String, dynamic>> recordScoreEvent({
    required String childId,
    required String activityId,
    required String activityTitle,
    required String activitySource, // preset | ai_generated | llg_assigned | migration
    required String skillDomain,
    required String difficulty,
    required int levelAtTime,
    required double rawScoreDelta,
    required Map<String, dynamic> feedback,
    String? submittedByParentId,
  }) async {
    if (childId.trim().isEmpty) {
      throw ArgumentError('childId cannot be empty');
    }
    if (skillDomain.trim().isEmpty) {
      throw ArgumentError('skillDomain cannot be empty');
    }
    final normalizedDomain = skillDomain.trim().toLowerCase();
    if (!DOMAINS.contains(normalizedDomain)) {
      throw ArgumentError('Invalid skill domain: $skillDomain');
    }
    if (rawScoreDelta.isNaN || rawScoreDelta.isInfinite) {
      throw ArgumentError('rawScoreDelta must be a finite number');
```

```
}

final weight = getWeight(activitySource, levelAtTime);
final appliedDelta = rawScoreDelta * weight;

// Secure flow: In production, route score events to the authoritative backend endpoint
// to calculate deltas and create ledger records server-side via Firebase Admin SDK.
if (_customFirestore == null) {
 try {
  final idToken = await FirebaseAuth.instance.currentUser?.getIdToken();
  if (idToken != null && idToken.isNotEmpty) {
   final uri = Uri.parse('$backendUrl/api/cases/record-activity-score');
   final client = _customHttpClient ?? http.Client();
   final response = await client.post(
    uri,
    headers: {
     'Content-Type': 'application/json',
     'Authorization': 'Bearer $idToken',
    },
    body: jsonEncode({
     'child_id': childId,
     'activity_id': activityId,
     'activity_title': activityTitle,
     'activity_source': activitySource,
     'skill_domain': normalizedDomain,
     'difficulty': difficulty,
     'child_response': feedback['childResponse']?.toString() ?? 'great',
     'engagement': feedback['engagement']?.toString() ?? 'high',
     'confidence': feedback['confidence']?.toString() ?? 'confident',
     'time_estimate': feedback['timeEstimate']?.toString(),
     'notes': feedback['notes']?.toString(),
    }),
   ).timeout(const Duration(seconds: 10));
```

```
        if (response.statusCode == 200 || response.statusCode == 201) {
          final data = jsonDecode(response.body) as Map<String, dynamic>;
          return {
            'eventId': data['event_id'] ?? 'server_evt_${DateTime.now().millisecondsSinceEpoch}',
            'scoreBefore': (data['score_before'] as num?)?.toDouble() ?? 0.0,
            'scoreAfter': (data['score_after'] as num?)?.toDouble() ?? 0.0,
            'rawDelta': rawScoreDelta,
            'appliedDelta': (data['applied_score_delta'] as num?)?.toDouble() ?? appliedDelta,
            'weight': (data['weight_applied'] as num?)?.toDouble() ?? weight,
            'source': activitySource,
            'isDomainMastered': data['is_domain_mastered'] ?? false,
            'isLevelCompleted': data['is_level_completed'] ?? false,
            'currentLevel': data['current_level'],
            'unlockedLevels': data['unlocked_levels'],
          };
        }
      }
    } catch (err) {
      debugPrint('⚠️ Backend growth endpoint call failed or offline, using fallback: $err');
    }
  }

  try {
    final profileRef = _firestore.collection('skillProfiles').doc(childId);
    final profileDoc = await profileRef.get();
    final profileData = profileDoc.data() ?? {};
    final scoreBefore = (profileData[normalizedDomain] is num)
        ? (profileData[normalizedDomain] as num).toDouble()
        : 0.0;
    final scoreAfter = (scoreBefore + appliedDelta).clamp(0.0, 100.0);
    if (scoreAfter < 0.0 || scoreAfter > 100.0 || scoreAfter.isNaN || scoreAfter.isInfinite) {
      throw ArgumentError('scoreAfter must be between 0 and 100');
    }
```

```
      // Immutable event document
      final eventRef = await _firestore.collection('scoreEvents').add({
        'childId': childId,
        'activityId': activityId,
        'activityTitle': activityTitle,
        'activitySource': activitySource,
        'skillDomain': normalizedDomain,
        'difficulty': difficulty,
        'levelAtTime': levelAtTime,
        'feedback': feedback,
        'rawScoreDelta': rawScoreDelta,
        'appliedScoreDelta': appliedDelta,
        'weightApplied': weight,
        'scoreBefore': scoreBefore,
        'scoreAfter': scoreAfter,
        'submittedByParentId': submittedByParentId,
        'createdAt': FieldValue.serverTimestamp(),
        'isImmutable': true,
      });

      // Profile cache (fast lookup recomputed directly from ledger delta)
      await profileRef.set({
        normalizedDomain: scoreAfter,
        'lastEventId': eventRef.id,
        'childId': childId,
        'updatedAt': FieldValue.serverTimestamp(),
      }, SetOptions(merge: true));

      debugPrint(
        '✅ Score event: $normalizedDomain $scoreBefore → $scoreAfter (Δ$appliedDelta
  w$weight source:$activitySource)',
      );

      return {
```

```
        'eventId': eventRef.id,
        'scoreBefore': scoreBefore,
        'scoreAfter': scoreAfter,
        'rawDelta': rawScoreDelta,
        'appliedDelta': appliedDelta,
        'weight': weight,
        'source': activitySource,
      };
    } catch (e, stack) {
      debugPrint('❌ Failed to record score event: $e\n$stack');
      rethrow;
    }
  }

  /// Full audit history for a child
  Future<List<Map<String, dynamic>>> getScoreHistory({
    required String childId,
    String? skillDomain,
  }) async {
    try {
      Query q = _firestore.collection('scoreEvents').where('childId', isEqualTo: childId);
      if (skillDomain != null && skillDomain.isNotEmpty) {
        q = q.where('skillDomain', isEqualTo: skillDomain.trim().toLowerCase());
      }
      final snap = await q.orderBy('createdAt', descending: true).get();
      return snap.docs
          .map((d) => {'id': d.id, ...d.data() as Map<String, dynamic>})
          .toList();
    } catch (e, stack) {
      debugPrint('❌ Failed to get score history: $e\n$stack');
      rethrow;
    }
  }
```

```
/// Recompute profile from scratch (repair tool + verification)
Future<Map<String, double>> recomputeProfile(String childId) async {
  try {
    final events = await getScoreHistory(childId: childId);
    // Reverse to process chronologically if ordered descending
    final chronological = events.reversed.toList();

    final Map<String, double> totals = {
      'cognitive': 0.0,
      'language': 0.0,
      'motor': 0.0,
      'social': 0.0,
      'emotional': 0.0,
      'creative': 0.0,
    };

    for (var e in chronological) {
      final rawDomain = e['skillDomain'] as String?;
      if (rawDomain == null) continue;
      final domain = rawDomain.trim().toLowerCase();
      if (totals.containsKey(domain)) {
        final delta = (e['appliedScoreDelta'] is num)
            ? (e['appliedScoreDelta'] as num).toDouble()
            : 0.0;
        totals[domain] = (totals[domain]! + delta).clamp(0.0, 100.0);
      }
    }

    await _firestore.collection('skillProfiles').doc(childId).set({
      ...totals,
      'childId': childId,
      'updatedAt': FieldValue.serverTimestamp(),
    }, SetOptions(merge: true));
```

```
    debugPrint('✅ Recomputed profile for $childId: $totals');
    return totals;
  } catch (e, stack) {
    debugPrint('❌ Failed to recompute profile: $e\n$stack');
    rethrow;
  }
}

/// Checkpoint 2: Data Migration for Existing Children
/// Reads existing skillProfiles and writes traceable baseline scoreEvents
Future<int> migrateExistingSkillProfiles() async {
  debugPrint('🔄 Starting Checkpoint 2 Migration for existing skill profiles...');
  int migratedCount = 0;

  try {
    final profilesSnap = await _firestore.collection('skillProfiles').get();

    for (var profileDoc in profilesSnap.docs) {
      final childId = profileDoc.id;
      final data = profileDoc.data();

      // Check if baseline migration events already exist for this child
      final existingMigrationSnap = await _firestore
          .collection('scoreEvents')
          .where('childId', isEqualTo: childId)
          .where('activitySource', isEqualTo: 'migration')
          .limit(1)
          .get();

      if (existingMigrationSnap.docs.isNotEmpty) {
        debugPrint('ℹ️ Child $childId already has baseline migration events. Skipping.');
        continue;
      }
```

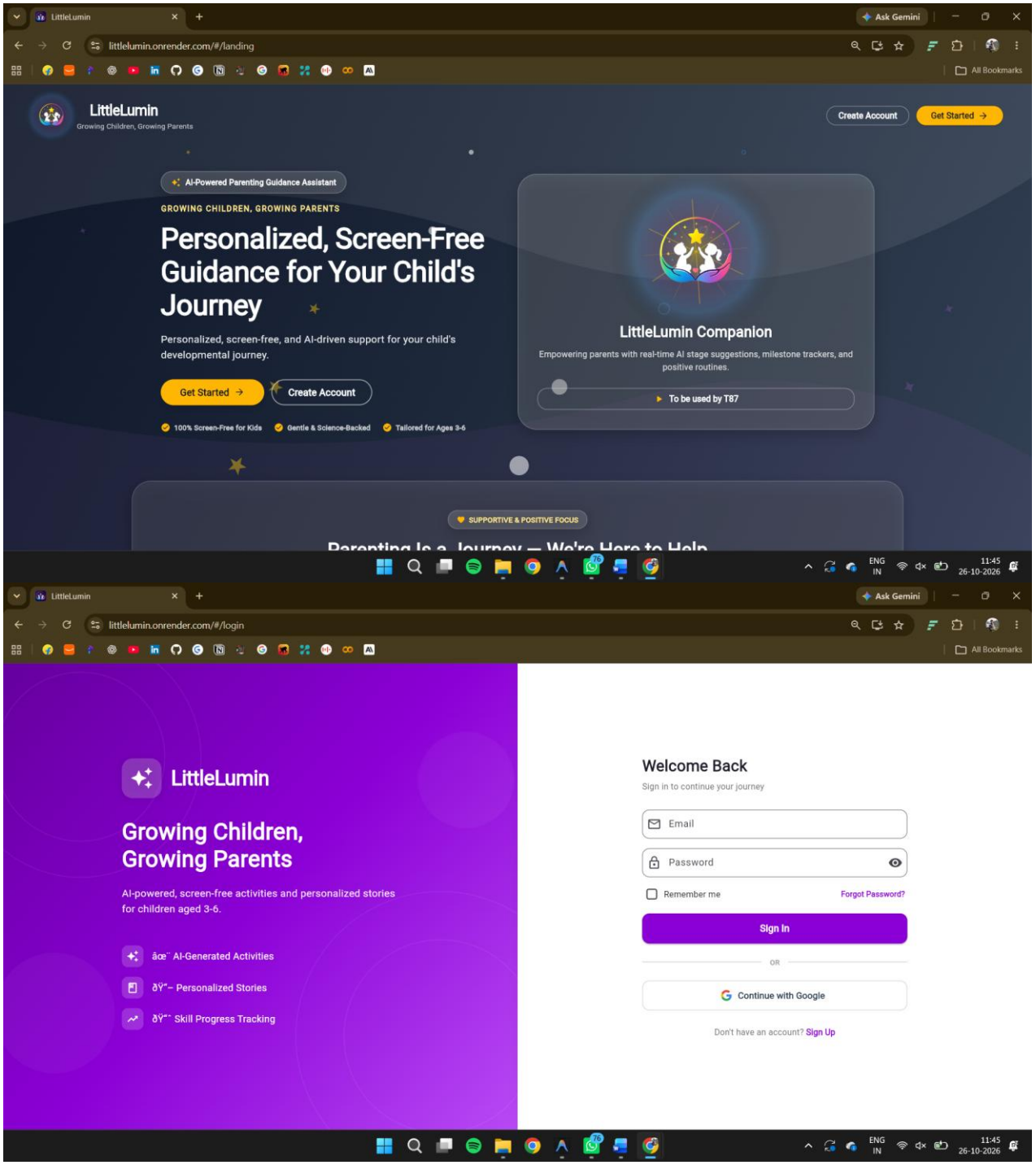
```
    bool hasMigratedDomain = false;
    for (var domain in DOMAINS) {
      final score = (data[domain] is num) ? (data[domain] as num).toDouble() : 0.0;
      if (score > 0) {
        await _firestore.collection('scoreEvents').add({
          'childId': childId,
          'activityId': 'baseline_migration_$domain',
          'activityTitle': 'Historical Baseline Score Migration ($domain)',
          'activitySource': 'migration',
          'skillDomain': domain,
          'difficulty': 'baseline',
          'levelAtTime': 1,
          'feedback': {'migration': true, 'initialScore': score},
          'rawScoreDelta': score,
          'appliedScoreDelta': score,
          'weightApplied': 1.0,
          'scoreBefore': 0.0,
          'scoreAfter': score,
          'createdAt': FieldValue.serverTimestamp(),
          'isImmutable': true,
        });
        hasMigratedDomain = true;
      }
    }

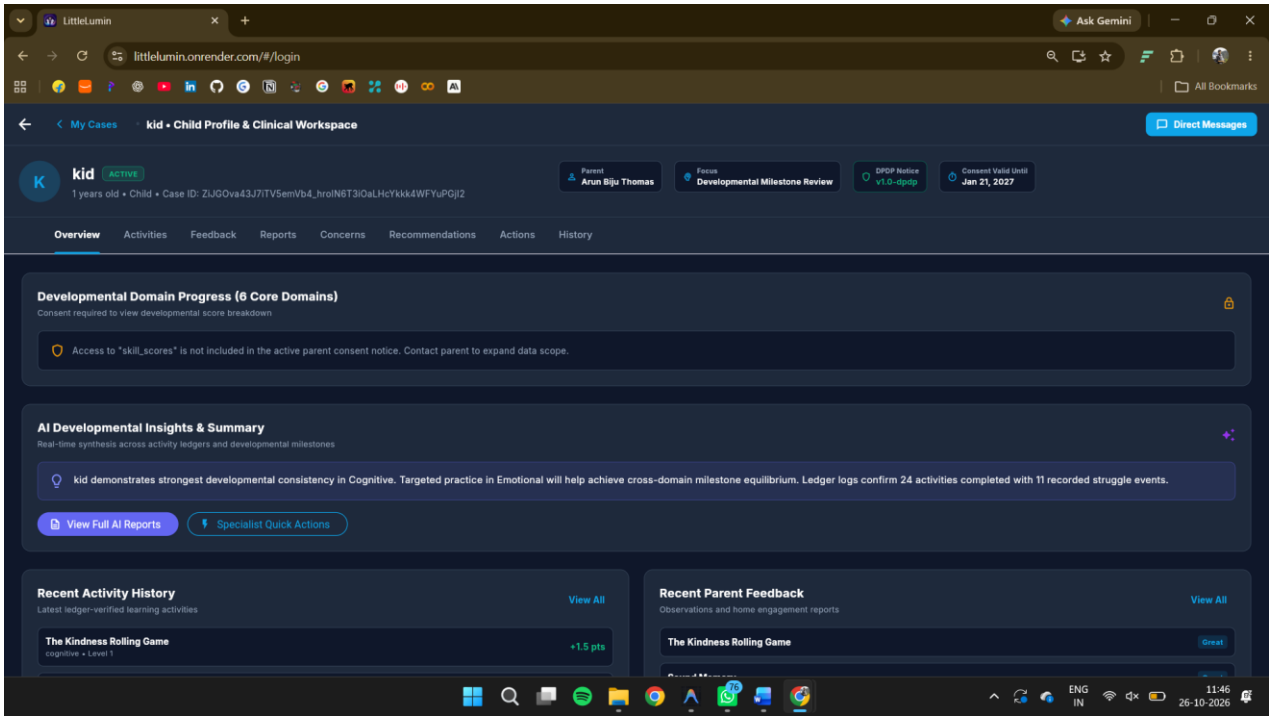
    if (hasMigratedDomain) {
      migratedCount++;
      debugPrint('✅ Migrated baseline scores for child $childId');
    }
  }

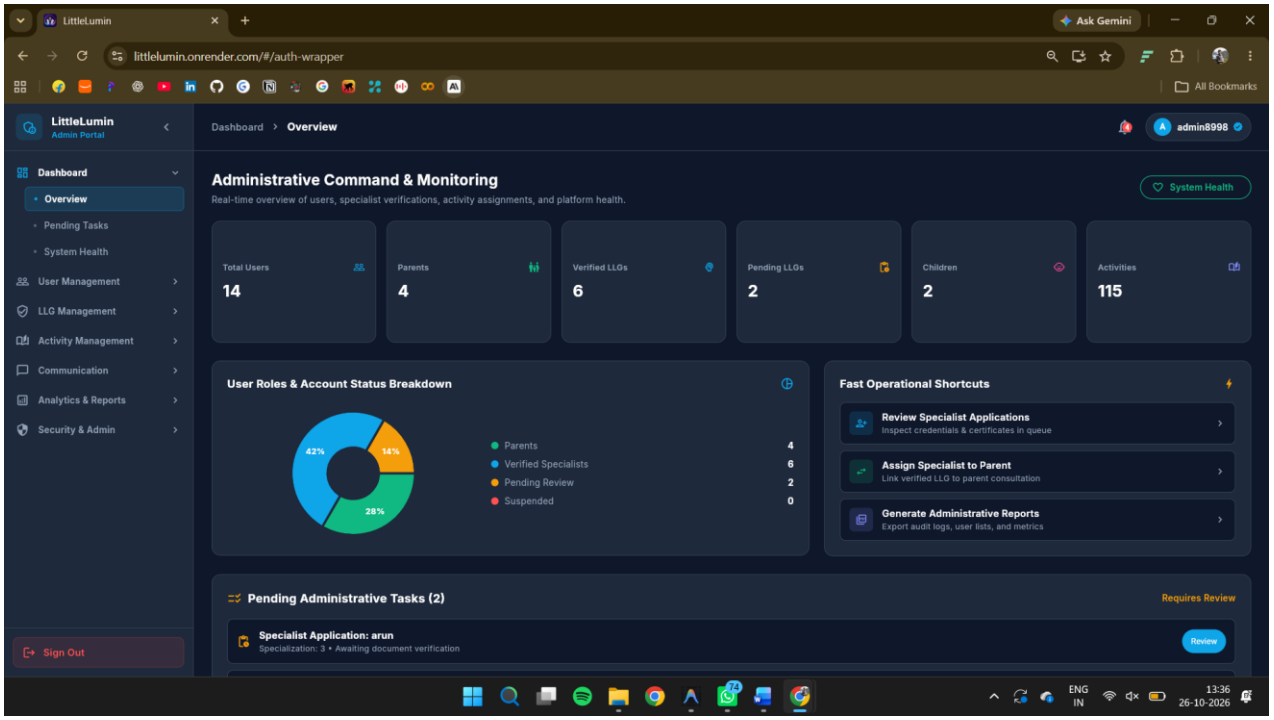
  debugPrint('🎉 Migration complete. Total children migrated: $migratedCount');
  return migratedCount;
```

```
    } catch (e, stack) {
      debugPrint('❌ Migration failed: $e\n$stack');
      rethrow;
    }
  }
}
```

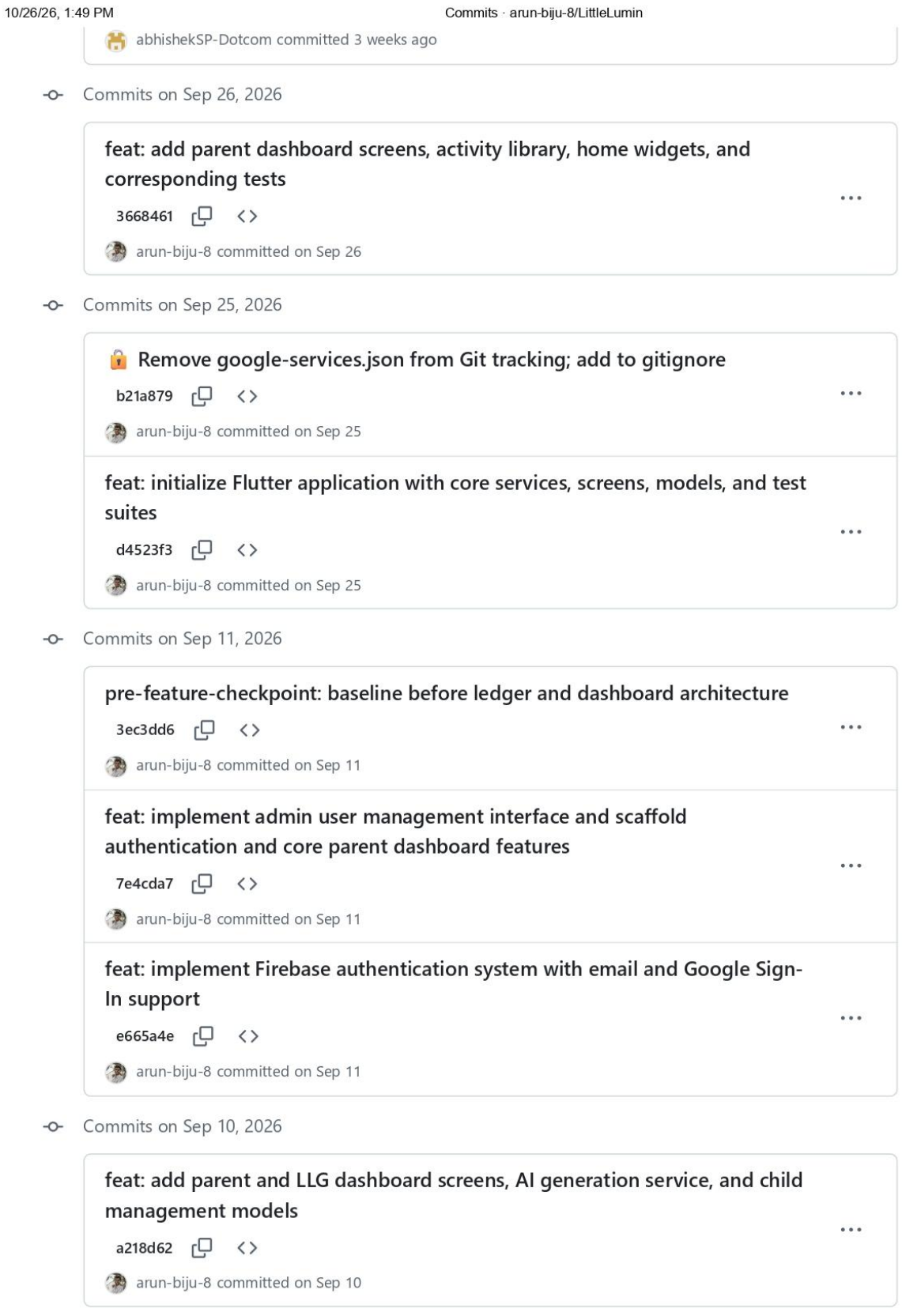
9.2 Screen Shots




LittleLumin
littlelumin.onrender.com/#/login
My Cases
kid • Child Profile & Clinical Workspace
Direct Messages
kid
ACTIVE
Overview
Activities
Feedback
Reports
Concerns
Recommendations
Actions
History
Developmental Domain Progress (6 Core Domains)
Consent required to view developmental score breakdown
Access to "skill_scores" is not included in the active parent consent notice. Contact parent to expand data scope.
AI Developmental Insights & Summary
Real-time synthesis across activity ledgers and developmental milestones
kid demonstrates strongest developmental consistency in Cognitive. Targeted practice in Emotional will help achieve cross-domain milestone equilibrium. Ledger logs confirm 24 activities completed with 11 recorded struggle events.
View Full AI Reports
Specialist Quick Actions
Recent Activity History
Latest ledger-verified learning activities
View All
The Kindness Rolling Game
+1.5 pts
Recent Parent Feedback
Observations and home engagement reports
View All
The Kindness Rolling Game
Great


LittleLumin
littlelumin.onrender.com/#/auth-wrapper
LittleLumin
Admin Portal
Dashboard
Overview
Pending Tasks
System Health
User Management
LLG Management
Activity Management
Communication
Analytics & Reports
Security & Admin
Sign Out
Dashboard › Overview
admin8998
Administrative Command & Monitoring
Real-time overview of users, specialist verifications, activity assignments, and platform health.
System Health
Total Users
14
Parents
4
Verified LLGs
6
Pending LLGs
2
Children
2
Activities
115
User Roles & Account Status Breakdown
42%
14%
28%
Parents 4
Verified Specialists 6
Pending Review 2
Suspended 0
Fast Operational Shortcuts
Review Specialist Applications
Inspect credentials & certificates in queue
Assign Specialist to Parent
Link verified LLG to parent consultation
Generate Administrative Reports
Export audit logs, user lists, and metrics
Pending Administrative Tasks (2)
Requires Review
Specialist Application: arun
Specialization: 3 • Awaiting document verification
Review

9.3 Git Log


10/26/26, 1:49 PM
Commits · arun-biju-8/LittleLumin
abhishekSP-Dotcom committed 3 weeks ago
Commits on Sep 26, 2026
feat: add parent dashboard screens, activity library, home widgets, and corresponding tests
3668461
arun-biju-8 committed on Sep 26
Commits on Sep 25, 2026
Remove google-services.json from Git tracking; add to gitignore
b21a879
arun-biju-8 committed on Sep 25
feat: initialize Flutter application with core services, screens, models, and test suites
d4523f3
arun-biju-8 committed on Sep 25
Commits on Sep 11, 2026
pre-feature-checkpoint: baseline before ledger and dashboard architecture
3ec3dd6
arun-biju-8 committed on Sep 11
feat: implement admin user management interface and scaffold authentication and core parent dashboard features
7e4cda7
arun-biju-8 committed on Sep 11
feat: implement Firebase authentication system with email and Google Sign-In support
e665a4e
arun-biju-8 committed on Sep 11
Commits on Sep 10, 2026
feat: add parent and LLG dashboard screens, AI generation service, and child management models
a218d62
arun-biju-8 committed on Sep 10
https://github.com/arun-biju-8/LittleLumin/commits/main/
4/5

arun-biju-8 committed 3 weeks ago

Rebuild web: Google Sign-In fix

82afe49

arun-biju-8 committed 3 weeks ago

Fix Google Sign-In on web with signInWithPopup

e8625b4

arun-biju-8 committed 3 weeks ago

Rebuild web: LLG Google sign-in

20be8bc

arun-biju-8 committed 3 weeks ago

Add Continue with Google to LLG application + fix merge writes

41681c5

arun-biju-8 committed 3 weeks ago

Deploy: prebuilt web output for Render

bb8f50f

arun-biju-8 committed 3 weeks ago

Commits on Oct 7, 2026

Unified AuthPage + .env cleanup

c113874

arun-biju-8 committed 3 weeks ago

Remove build artifacts and secrets from tracking

dc66487

arun-biju-8 committed 3 weeks ago

🔒 Remove .env from public web build

caaf664

arun-biju-8 committed 3 weeks ago

feat: add authentication and landing page screens for LittleLumin application

a46f6ab

abhishekSP-Dotcom committed 3 weeks ago

Add Flutter web build for deployment

b243432

arun-biju-8 committed 3 days ago

Make Parent Growth Portal easier to understand
b7bb510
arun-biju-8 committed 3 days ago

Fix parent portal navigation after login
3bb4da0
arun-biju-8 committed 3 days ago

Improve Parent and LLG collaboration portal
df38f68
arun-biju-8 committed 3 days ago

Add specialist support to Parent home
577db59
arun-biju-8 committed 3 days ago

Improve parent and LLG collaboration features
c38589f
arun-biju-8 committed 3 days ago

Add desktop parent growth portal
7664787
arun-biju-8 committed 3 days ago

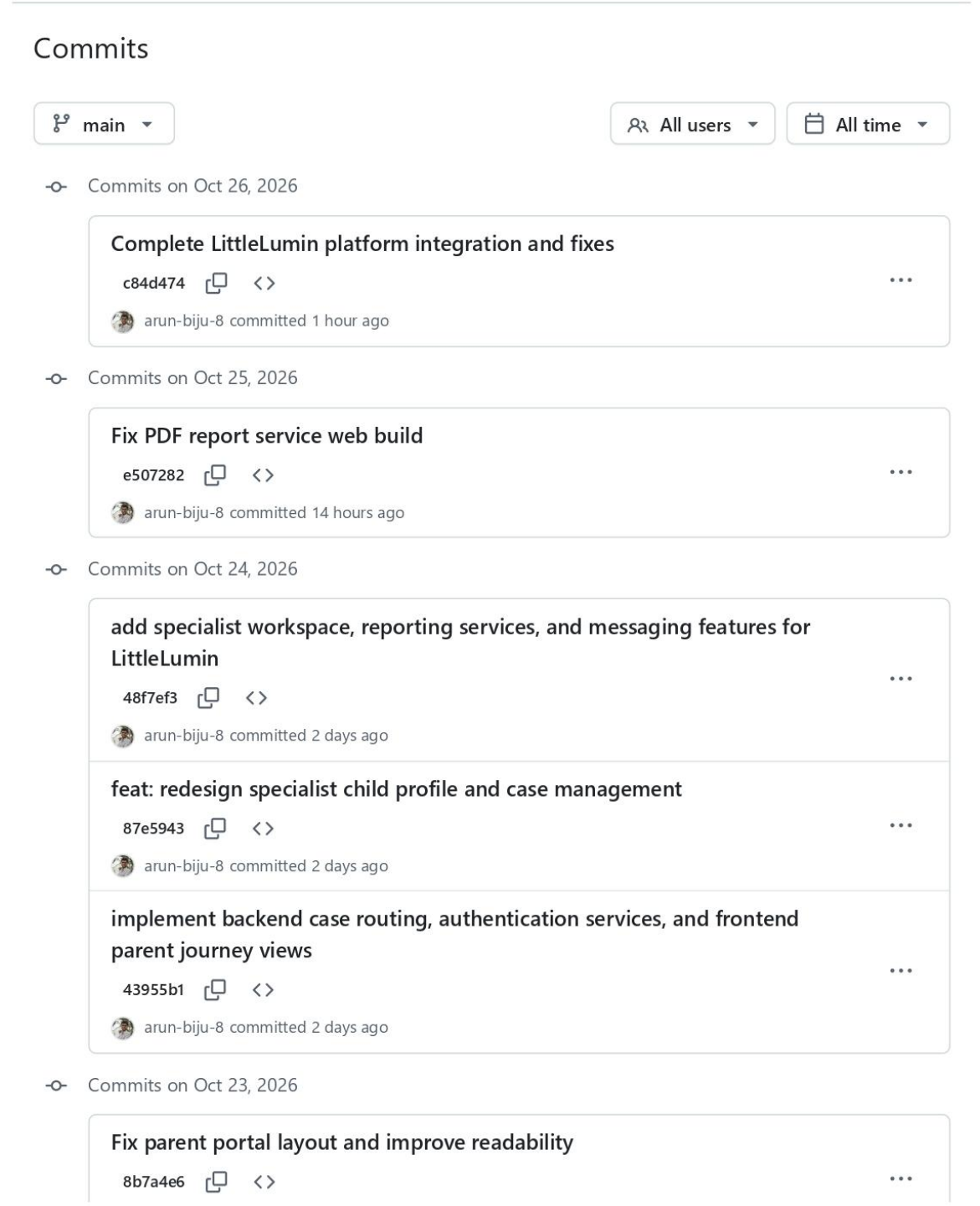
Improve LittleLumin security and learning experience
03a55f9
arun-biju-8 committed 3 days ago

Fix Flutter web build by removing missing env asset
39144e5
arun-biju-8 committed 4 days ago

Commits on Oct 8, 2026

Deploy: new Firebase Web API key
524b7f1
arun-biju-8 committed 3 weeks ago

Rebuild web with updated Firebase API key
bc1f739


Commits
main
All users
All time
Commits on Oct 26, 2026
Complete LittleLumin platform integration and fixes
c84d474
arun-biju-8 committed 1 hour ago
Commits on Oct 25, 2026
Fix PDF report service web build
e507282
arun-biju-8 committed 14 hours ago
Commits on Oct 24, 2026
add specialist workspace, reporting services, and messaging features for LittleLumin
48f7ef3
arun-biju-8 committed 2 days ago
feat: redesign specialist child profile and case management
87e5943
arun-biju-8 committed 2 days ago
implement backend case routing, authentication services, and frontend parent journey views
43955b1
arun-biju-8 committed 2 days ago
Commits on Oct 23, 2026
Fix parent portal layout and improve readability
8b7a4e6
https://github.com/arun-biju-8/LittleLumin/commits/main/
1/5


10/26/26, 1:49 PM
Commits · arun-biju-8/LittleLumin
Commits on Sep 9, 2026
Fix generate_story argument signature to accept topic_or_moral
a9269ad
arun-biju-8 committed on Sep 9
Add debug logging to generate_story method
1e16fb9
arun-biju-8 committed on Sep 9
Previous
Next